

R Basics

Haewon Lee

2025-03-17

Table of contents

I Basics

I.A R data types

I.A.1 variable type 종류

- numeric, complex

```
rm(list=ls())
(numeric_value <- pi)
```

```
[1] 3.141593
```

```
(1+sqrt(2)*1i)*(1-sqrt(2)*1i) # complex 연산
```

```
[1] 3+0i
```

```
options(digits=20) # 20자리 표현 (default=7)
pi
```

```
[1] 3.141592653589793116
```

- integer

```
options(digits=10)
(integer_value <- 42L) #반드시 정수라야 하는 경우에는 뒤에 L을 붙인다.
```

```
[1] 42
```

```
typeof(1+1); typeof(1L+1L); typeof(1:3)
```

```
[1] "double"
```

```
[1] "integer"
```

```
[1] "integer"
```

- logical 논리 값 TRUE | FALSE 두가지 값중 하나를 갖는다. 비교를 할 때에는 == 를 사용 (=는 input 명령이 됨)

```
1<0 ; 1>0 ; 1<"a"; "a">"A"; 3==6 #한줄에 여러 명령을 쓸 때에는 semicolon ; 로 구분
```

```
[1] FALSE
```

```
[1] TRUE
```

```
[1] TRUE
```

```
[1] FALSE
```

```
[1] FALSE
```

- character : “abc”, “a”, “a123xz” 등 quotation mark로 된 문자열 ” = \” 로 표기 줄바꿈 = \n

```
letters[5:10]; paste("ab","cde", sep = "")
```

```
[1] "e" "f" "g" "h" "i" "j"
```

```
[1] "abcde"
```

```
as.character(345); as.numeric("23.5")
```

```
[1] "345"
```

```
[1] 23.5
```

```
sub("a","x", "father and grandpa"); gsub("a","x", "father and grandpa")
```

```
[1] "fxther and grandpa"
```

```
[1] "fxther xnd grxndpx"
```

```
(ex2 <- 'The "R" project for statistical computing')
```

```
[1] "The \"R\" project for statistical computing"
```

- Escape characters in R :

- \t Insert a tab in the text at this point.
- \b Insert a backspace in the text at this point.
- \n Insert a newline in the text at this point.
- \r Insert a carriage return in the text at this point.
- \f Insert a formfeed in the text at this point.
- \s Insert a space in the text at this point.
- \' Insert a single quote character in the text at this point.
- \" Insert a double quote character in the text at this point.
- \\ Insert a backslash character in the text at this point.

- raw : used for binary data

- time : r 에서는 시간을 다루는 방법이 매우 다양함 POSIX시간변수는 복잡한 list 형태로 되어 있음

```
(time1 <- as.POSIXlt("1960-01-01")); class(time1); typeof(time1)
```

```
[1] "1960-01-01 KST"
```

```
[1] "POSIXlt" "POSIXt"
```

```
[1] "list"
```

```
first <- "2022-08-20 08:15:22" ; second <- "2022-01-01 20:04:48"
difftime(first, second); difftime(first, second, units = "hours")
```

Time difference of 230.507338 days

Time difference of 5532.176111 hours

```
first2 <- as.POSIXlt(first); second2 <- as.POSIXlt(second)
second2 - first2
```

Time difference of -230.507338 days

```
## difftime(first, second, units = "months")
## match.arg(units)에서 다음과 같은 에러가 발생했습니다:
## 'arg' should be one of "auto", "secs", "mins", "hours", "days", "weeks"
```

시간과 날짜를 다루는 방법에 대해서는 뒤에 Section ?? 에서 다시 다루도록 한다.

I.A.2 data types

- vector : R에서는 모든 변수가 벡터 (열) 로 되어 있다. 다음 연산결과를 예상해 보시오

```
1:3 + 2:4 ; 1:10 + 1:2
```

```
[1] 3 5 7
```

```
[1] 2 4 4 6 6 8 8 10 10 12
```

```
paste(LETTERS[1:10], 1:3, sep = "-"); paste(LETTERS[1:3], 1:10)
```

```
[1] "A-1" "B-2" "C-3" "D-1" "E-2" "F-3" "G-1" "H-2" "I-3" "J-1"
```

```
[1] "A 1" "B 2" "C 3" "A 4" "B 5" "C 6" "A 7" "B 8" "C 9" "A 10"
```

vector의 특징은 모든 요소가 단일한 것이라는 점이다. NA 값을 제외하고는 모든 요소가 같아야 하기 때문에 서로 다른 성질의 것을 입력하게 되면 에러가 생기거나 변형된다.

```
c(1,2,3); c(1,2,3,"a")
```

```
[1] 1 2 3
```

```
[1] "1" "2" "3" "a"
```

vector 연산은 순환연산이다. 6개의 원소를 가진 벡터와 3개의 원소를 가진 벡터를 연산하면 갯수가 안 맞는데 이런 경우 3개의 원소를 가진 벡터를 처음부터 다시 한번 더 사용한다. 만약 6개 짜리 벡터와 4개짜리 벡터를 계산하면 어떻게 될까?

```
x6 <- 2:7
y4 <- 1:4
z1 <- x6 %% y4
z1
```

```
[1] 0 1 1 1 0 1
```

계산이 정상적으로 된다. 다만 “경고: 두 객체의 길이가 서로 배수관계에 있지 않습니다” 라는 경고가 나온다. 4개짜리 벡터를 다시 사용하는데 처음 두개만 사용하고 중단하기 때문이다.

이렇듯 벡터의 길이가 달라도 연산이 에러를 내면서 멈추거나 하지 않고 경고만 내고 정상적으로 되는 것처럼 보인다는 사실을 잘 알고 있어야 한다.

- array : multidimensional vector

```
(arr1 <- array(data=1:90, dim = c(6,5,3))) # 3Dimensional array
```

```
, , 1
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	1	7	13	19	25
[2,]	2	8	14	20	26
[3,]	3	9	15	21	27
[4,]	4	10	16	22	28
[5,]	5	11	17	23	29
[6,]	6	12	18	24	30

```
, , 2
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	31	37	43	49	55
[2,]	32	38	44	50	56
[3,]	33	39	45	51	57
[4,]	34	40	46	52	58
[5,]	35	41	47	53	59
[6,]	36	42	48	54	60

```
, , 3
```

```

      [,1] [,2] [,3] [,4] [,5]
[1,]  61   67   73   79   85
[2,]  62   68   74   80   86
[3,]  63   69   75   81   87
[4,]  64   70   76   82   88
[5,]  65   71   77   83   89
[6,]  66   72   78   84   90

```

```
arr1[6,4,2] # 3Dimensional indexing
```

```
[1] 54
```

```
which(arr1==54, arr.ind = TRUE )
```

```

      dim1 dim2 dim3
[1,]    6    4    2

```

- matrix : 2dimensional vector

```

matrix(data = c(3,4,5,6,7,8),
       nrow=2,
       ncol=3, # nrow=2 하나만 지정해도 ncol=3은 내부적으로 결정됨
       byrow = TRUE, # data assign 하는 방향
       dimnames = list(c("pt_1", "pt_2"), # row names
                       c("var1", "var2", "var3")) # col names
)

```

```

      var1 var2 var3
pt_1    3    4    5
pt_2    6    7    8

```

```

x <- 2:9 ; names(x) <- x # x의 이름을 부여
x %o% x # = outer function : outer(x,x, FUN="*")

```

```

  2  3  4  5  6  7  8  9
2  4  6  8 10 12 14 16 18
3  6  9 12 15 18 21 24 27
4  8 12 16 20 24 28 32 36
5 10 15 20 25 30 35 40 45
6 12 18 24 30 36 42 48 54
7 14 21 28 35 42 49 56 63
8 16 24 32 40 48 56 64 72
9 18 27 36 45 54 63 72 81

```

- data frame : vector를 구성요소로 한 list의 형태 (외형적으로 보면 2dimension으로 보인다)
dataframe의 구성요소는 vector들 (각각의 vector는 동일한 데이터 타입이라야 함)

```
## dataframe 만들기
df1 <- data.frame( col1 = c("A", "B", "Anyone", "None"),
                   col2 = c(160, 170, 180, 200),
                   col3 = c(TRUE, FALSE, FALSE, TRUE)
                   )

df1
```

```
   col1 col2 col3
1     A  160 TRUE
2     B  170 FALSE
3 Anyone  180 FALSE
4    None  200  TRUE
```

데이터프레임 이름 <- data.frame(컬럼이름= c(data_1, ..., data_n), ...) 이런 형식으로 데이터 프레임 만들 수 있다. 데이터프레임이 R의 기본적인 데이터 양식이기 때문에 이를 다루는 방법이 다양하게 존재함

```
## dataframe cell 찾기
df1[3,2] #3행 2열의 데이터
```

```
[1] 180
```

```
## column 이름으로 찾기
df1$col1 ; df1[, "col1"]; df1["col1"] ### df1의 col1 열을 찾는 방법들
```

```
[1] "A"      "B"      "Anyone" "None"
```

```
[1] "A"      "B"      "Anyone" "None"
```

```
   col1
1     A
2     B
3 Anyone
4    None
```

```
df1[,1]
```

```
[1] "A"      "B"      "Anyone" "None"
```

- list : R에만 있는 독특한 데이터타입이다. 이것은 모든 데이터 타입을 담을 수 있는 형태이고 자료의 길이가 달라도 같이 담을 수가 있게 되어 있다. 또한 리스트 속에 리스트를 넣을 수 있기에 다단계로 nesting 되는 구조로 만들 수 있다.

```
sample_list <- list(data1=df1, data2 = arr1, data3 = x%o%x)
str(sample_list)
```

```
List of 3
 $ data1:'data.frame': 4 obs. of 3 variables:
  ..$ col1: chr [1:4] "A" "B" "Anyone" "None"
  ..$ col2: num [1:4] 160 170 180 200
  ..$ col3: logi [1:4] TRUE FALSE FALSE TRUE
 $ data2: int [1:6, 1:5, 1:3] 1 2 3 4 5 6 7 8 9 10 ...
 $ data3: num [1:8, 1:8] 4 6 8 10 12 14 16 18 6 9 ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : chr [1:8] "2" "3" "4" "5" ...
  .. ..$ : chr [1:8] "2" "3" "4" "5" ...
```

```
sample_list$data1
```

```
      col1 col2 col3
1       A  160  TRUE
2       B  170 FALSE
3 Anyone  180 FALSE
4   None  200  TRUE
```

```
sample_list$data1[,3]
```

```
[1] TRUE FALSE FALSE TRUE
```

I.A.3 Operators 연산자

일반 산수연산자

- + - * /
- `^` or `**` : exponential
- `%%` : modulus 나머지 연산자
- `%/%` : integer division 몫을 구하기

논리연산자

operator	description	operator	description
<	less than	>	greater than
<=	less than or equal to	>=	greater than or equal to
==	exactly equal to	!=	not equal to
x & y	x AND y	x y	x OR y
!x	NOT x	xor(x,y)	x XOR y exclusive or
&&	AND : &와 같으나 한개만 연산		OR : 와 같으나 한개만 연산

Pipe 연산자

%>% |>

R에서는 한번의 명령을 실행한 것을 다음으로 넘길 때에 pipe 연산자라는 것을 사용한다.

Ctrl+Shift+m (windows), cmd+Shift+m (MAC OS) 단축키

함수의 결과물이 다른 함수에 또 사용되고, 이 결과물이 다시 또 다른 함수에 사용되고.... 이런 식으로 진행될 때에는 이러한 pipe 연산자를 사용하는 것이 매우 편리하다.

magrittr 패키지에 포함된 연산자이며 dplyr 또는 tidyverse 사용시에도 자동으로 로드된다.

최근에는 이러한 pipe 연산자의 내재화에 대한 요구가 많아져서 R 4.1 버전부터 도입되기 시작하였으며 4.3 버전에서는 모든 기능이 업데이트 되었다. 내재화된 pipe 연산자는 |> 이며 사용법은 %>% 와 약간 다르다.

내재화된 pipe 연산자 |>에서는 왼쪽의 객체를 오른쪽 함수로 전달할 때 인수의 위치를 반드시 전달해야 한다. %>%에서는 인수의 위치가 명확하여 전달할 필요가 없으면 없어도 되며 필요한 경우 period . 을 사용하는데 |>에서는 underscore _를 사용한다. %>%는 함수 호출에 괄호를 생략할 수 있지만 |>는 괄호를 생략할 수 없다. - 저자는 아직 %>% 사용함

```
1 library(magrittr) # %>% 연산자를쓰기위해서로드 , |> 연산자는패키지가필요없다
2 round(exp(diff(log(x6))), 2)
3 x6 %>% log %>% diff %>% exp %>% round(., 2)
4 x6 |> log() |> diff() |> exp() |> round(x=_, 2) # x= argument 이름을명시해야
```

행렬 연산자

R에서는 행렬도 벡터로 간주되기 때문에 덧셈이나 뺄셈 또는 단순한 스칼라곱의 경우에는 벡터 연산과 같지만, 행렬의 곱에는 일반적인 곱셈연산자 *를 사용하면 벡터연산 결과만 나오므로 안된다. 수학적인 행렬의 연산을 하기 위해서는 아래의 연산자들을 사용해야 한다.

operator	description	operator	description
%*%	행렬곱	%o%	vector 외적 $\otimes$
t()	transposition	solve(A)	A의 역행렬
solve(A,b)	$Ax=b$ 를 만족하는 x를 구함	det(A)	A의 행렬식 $ A $
eigen(A)	고유값과 고유벡터	tensor(A,B)	텐서곱

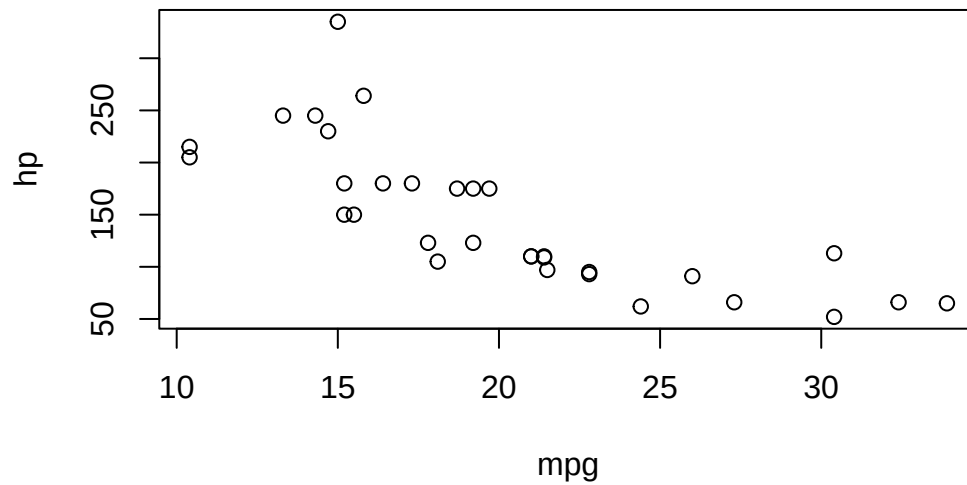
특수 연산자

A %in% B : A 중에서 B 벡터 내 원소의 값이 포함된 것이 있는지 알려줌 (logical value)

%<% : pipe 연산자로 연산도 이어서 하고 그 값을 원래의 변수에 다시 덮어쓰기

%%\$% : with() 함수와 동일하게 데이터프레임을 고정하고 컬럼변수에 접근할 수 있도록 해줌

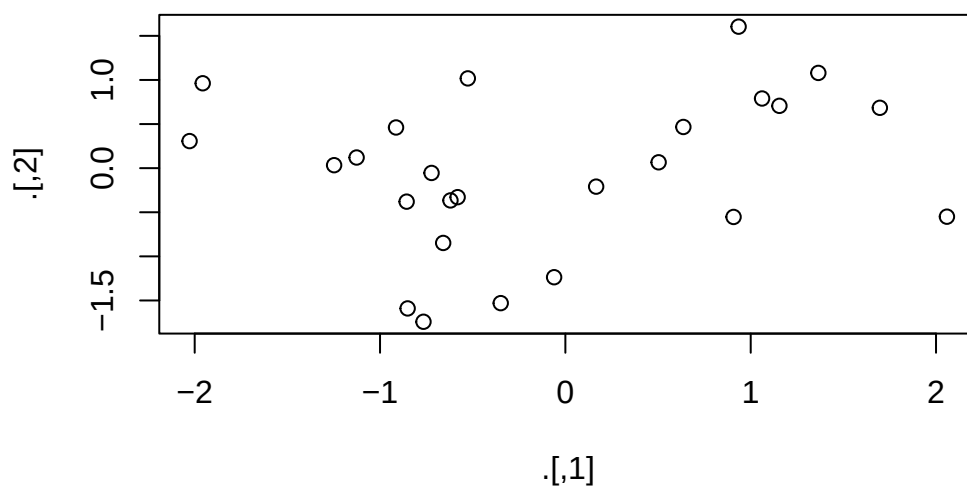
```
library(magrittr)
with(mtcars, plot(mpg,hp))
```



```
mtcars %$% plot(mpg, hp)
```

%T>% : Tee pipe 연산자 - 왼쪽의 결과값을 다시 그대로 리턴해서 사용 plot()과 같이 결과값이 없는 경우 사용함

```
library(magrittr)
library(dplyr)
#rnorm(50) %>%
# matrix(ncol=2) %>%
# plot() %>%
# colSums() ### Error 발생 = plot 함수에서 값을 전달하지 않음
rnorm(50) %>%
  matrix(ncol=2) %T>%
  plot() %>%
  colSums()
```



```
[1] -2.778836686 -1.085720426
```

%*t% : Matrix product A %*% t(B)

%t*% : Matrix product t(A) %*% B
 %t*t% : Matrix product t(A) %*% t(B)
 outer(x,y,FUN) :

I.B Built-in Data sets in R

I.B.1 소개 및 개요

R에는 내장된 데이터셋이 있다. 테스트용, 교육용 및 연습용으로 이러한 데이터셋을 사용하면 좋다.

- 사용법

```
data("volcano") ## built-in dataset 중에서 volcano 사용
library(survival)
data(package="survival") ## survival package에 어떤 데이터 세트들이 있는지 확인
data(cancer) ## data(cancer, package="survival") 와 같이 사용해도 된다.
str(lung) ## cancer dataset에는 다양한 암종류의 생존분석용 데이터가 들어가 있다.
```

```
'data.frame': 228 obs. of 10 variables:
 $ inst      : num  3 3 3 5 1 12 7 11 1 7 ...
 $ time      : num  306 455 1010 210 883 ...
 $ status    : num  2 2 1 2 2 1 2 2 2 2 ...
 $ age       : num  74 68 56 57 60 74 68 71 53 61 ...
 $ sex       : num  1 1 1 1 1 1 2 2 1 1 ...
 $ ph.ecog   : num  1 0 0 1 0 1 2 2 1 2 ...
 $ ph.karno  : num  90 90 90 90 100 50 70 60 70 70 ...
 $ pat.karno : num  100 90 90 60 90 80 60 80 80 70 ...
 $ meal.cal  : num  1175 1225 NA 1150 NA ...
 $ wt.loss   : num  NA 15 15 11 0 0 10 1 16 34 ...
```

- rotterdam : breast cancer dataset in survival package

```
library(moonBook)
mytable(grade~. , data=rotterdam)
```

Descriptive Statistics by 'grade'			
	2	3	p
	(N=794)	(N=2188)	
pid	1328.4 ± 865.1	1569.0 ± 860.9	0.000
year	1987.9 ± 3.1	1988.3 ± 3.0	0.004
age	54.4 ± 12.7	55.3 ± 13.1	0.086
meno			0.000
- 0	392 (49.4%)	920 (42.0%)	
- 1	402 (50.6%)	1268 (58.0%)	
size			0.000
- <=20	462 (58.2%)	925 (42.3%)	

- 20-50	290 (36.5%)	1001 (45.7%)	
- >50	42 (5.3%)	262 (12.0%)	
nodes	2.0 ± 3.7	3.0 ± 4.6	0.000
pgr	236.2 ± 385.8	134.9 ± 242.8	0.000
er	179.8 ± 291.9	161.8 ± 265.0	0.127
hormon			0.000
- 0	735 (92.6%)	1908 (87.2%)	
- 1	59 (7.4%)	280 (12.8%)	
chemo			1.000
- 0	640 (80.6%)	1762 (80.5%)	
- 1	154 (19.4%)	426 (19.5%)	
rtime	2458.3 ± 1408.6	1967.1 ± 1370.8	0.000
recur			0.000
- 0	480 (60.5%)	984 (45.0%)	
- 1	314 (39.5%)	1204 (55.0%)	
dtime	2908.9 ± 1278.6	2495.2 ± 1287.8	0.000
death			0.000
- 0	532 (67.0%)	1178 (53.8%)	
- 1	262 (33.0%)	1010 (46.2%)	

```
mytable(grade~. , data=rotterdam) %>% mylatex() %>% cat
```

```
## LaTeX을 이용하여 깔끔한 논문형식의 테이블을 만들 수 있다.
```

Descriptive Statistics by grade			
	2 (N=794)	3 (N=2188)	P
pid	1328.4 $\pm$ 865.1	1569.0 $\pm$ 860.9	0.000
year	1987.9 $\pm$ 3.1	1988.3 $\pm$ 3.0	0.004
age	54.4 $\pm$ 12.7	55.3 $\pm$ 13.1	0.086
meno			0.000
- 0	392 (49.4%)	920 (42.0%)	
- 1	402 (50.6%)	1268 (58.0%)	
size			0.000
- ≤ 20	462 (58.2%)	925 (42.3%)	
- 20-50	290 (36.5%)	1001 (45.7%)	
- > 50	42 (5.3%)	262 (12.0%)	
nodes	2.0 $\pm$ 3.7	3.0 $\pm$ 4.6	0.000
pgr	236.2 $\pm$ 385.8	134.9 $\pm$ 242.8	0.000
er	179.8 $\pm$ 291.9	161.8 $\pm$ 265.0	0.127
hormon			0.000
- 0	735 (92.6%)	1908 (87.2%)	
- 1	59 (7.4%)	280 (12.8%)	
chemo			1.000
- 0	640 (80.6%)	1762 (80.5%)	
- 1	154 (19.4%)	426 (19.5%)	
rtime	2458.3 $\pm$ 1408.6	1967.1 $\pm$ 1370.8	0.000
recur			0.000
- 0	480 (60.5%)	984 (45.0%)	
- 1	314 (39.5%)	1204 (55.0%)	
dtime	2908.9 $\pm$ 1278.6	2495.2 $\pm$ 1287.8	0.000
death			0.000
- 0	532 (67.0%)	1178 (53.8%)	
- 1	262 (33.0%)	1010 (46.2%)	

I.C Basic statistics functions

I.C.1 *t*-test

R function : `t.test` - 평균이 서로 다른지 비교, 통계적 유의성 검정

option arguments : `alternative = c("two.sided", "less", "greater")`, `formula` (종속변수~ 독립변수)

help files : `?t.test` 를 치면 함수의 argument, values(results), detail에 대해서 설명이 나옴

```
group1 <- rotterdam[ rotterdam$grade == 2, "age"]
group2 <- rotterdam[ rotterdam$grade != 2, "age"]
t.test(group1, group2) ## unmatched 임의의 두개의 vector로 비교
```

Welch Two Sample t-test

data: group1 and group2

t = -1.7436947, df = 1444.4213, p-value = 0.08142509

alternative hypothesis: true difference in means is not equal to 0

95 percent confidence interval:

-1.9599008823 0.1152640033

sample estimates:

mean of x mean of y

54.38161209 55.30393053

```
t.test(age~meno,data=rotterdam) ## matched 한개의 데이터프레임에서 paired t-test
```

Welch Two Sample t-test

data: age by meno

t = -76.545414, df = 2972.8397, p-value < 2.2204e-16

alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0

95 percent confidence interval:

-21.53192159 -20.45636342

sample estimates:

mean in group 0 mean in group 1

43.30106707 64.29520958

I.C.2 χ^2 (chi-square) test

R function : `chisq.test` - 범주형 변수의 각 그룹별 비율이 같은지 검정 (목적에 따라서 적합도, 동질성, 독립성 등을 검정가능)

```
table(rotterdam[,c("hormon","size")])
```

	size		
hormon	<=20	20-50	>50
0	1283	1119	241
1	104	172	63

```
chisq.test(table(rotterdam[,c("hormon","size")]), correct = TRUE)
```

Pearson's Chi-squared test

```
data: table(rotterdam[, c("hormon", "size")])
X-squared = 51.920064, df = 2, p-value = 5.317424e-12
```

```
chisq.test(rotterdam$hormon, rotterdam$chemo)
```

Pearson's Chi-squared test with Yates' continuity correction

```
data: rotterdam$hormon and rotterdam$chemo
X-squared = 29.771106, df = 1, p-value = 4.861842e-08
```

```
x <- matrix(c(12, 5, 7, 7), ncol = 2) ## matrix를 만들어서 검정하는 방법
x
```

```
      [,1] [,2]
[1,]   12   7
[2,]    5   7
```

```
chisq.test(x)$p.value ## chisq test의 결과물은 list이다 여기서 p.value 부분만 출력
```

```
[1] 0.4233054243
```

```
chisq.test(x, simulate.p.value = TRUE, B = 10000)$p.value
```

```
[1] 0.2832716728
```

I.C.3 generalized linear regression and Loess smoothing (LOcal regrESSion)

R function : glm (generalized linear models) 다중 선형회귀

```
rm(list=ls())
library(ggplot2)
data(economics, package="ggplot2")
economics$index <- 1:nrow(economics) # create index variable
glm_model1 <- glm(psavert~pop, data = economics)
summary(glm_model1)
```

Call:

```
glm(formula = psavert ~ pop, data = economics)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	2.594603e+01	4.811677e-01	53.92305	< 2.22e-16 ***
pop	-6.757974e-05	1.852367e-06	-36.48290	< 2.22e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 2.645600898)

Null deviance: 5034.5843 on 573 degrees of freedom
 Residual deviance: 1513.2837 on 572 degrees of freedom
 AIC: 2191.3815

Number of Fisher Scoring iterations: 2

```
anova(glm_model1)
```

Analysis of Deviance Table

Model: gaussian, link: identity

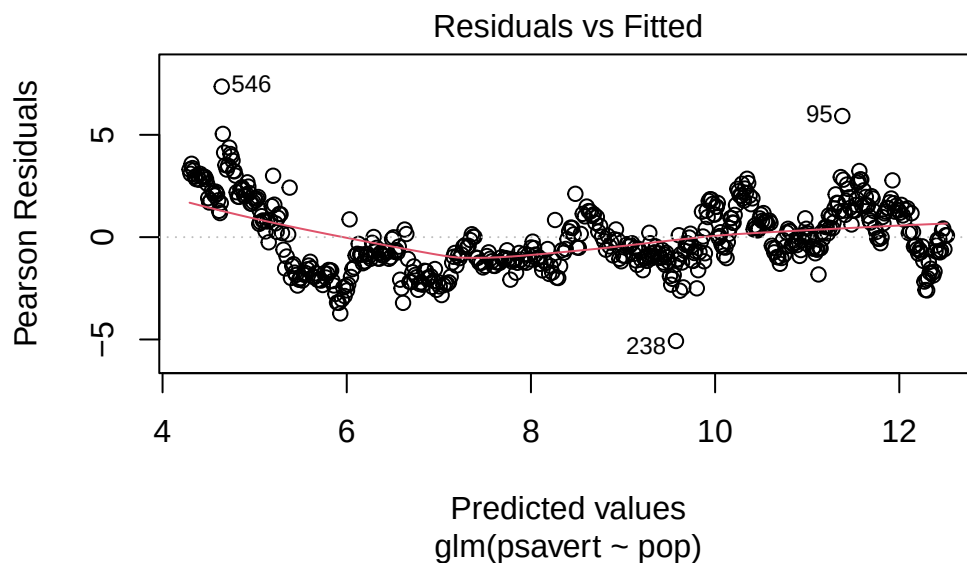
Response: psavert

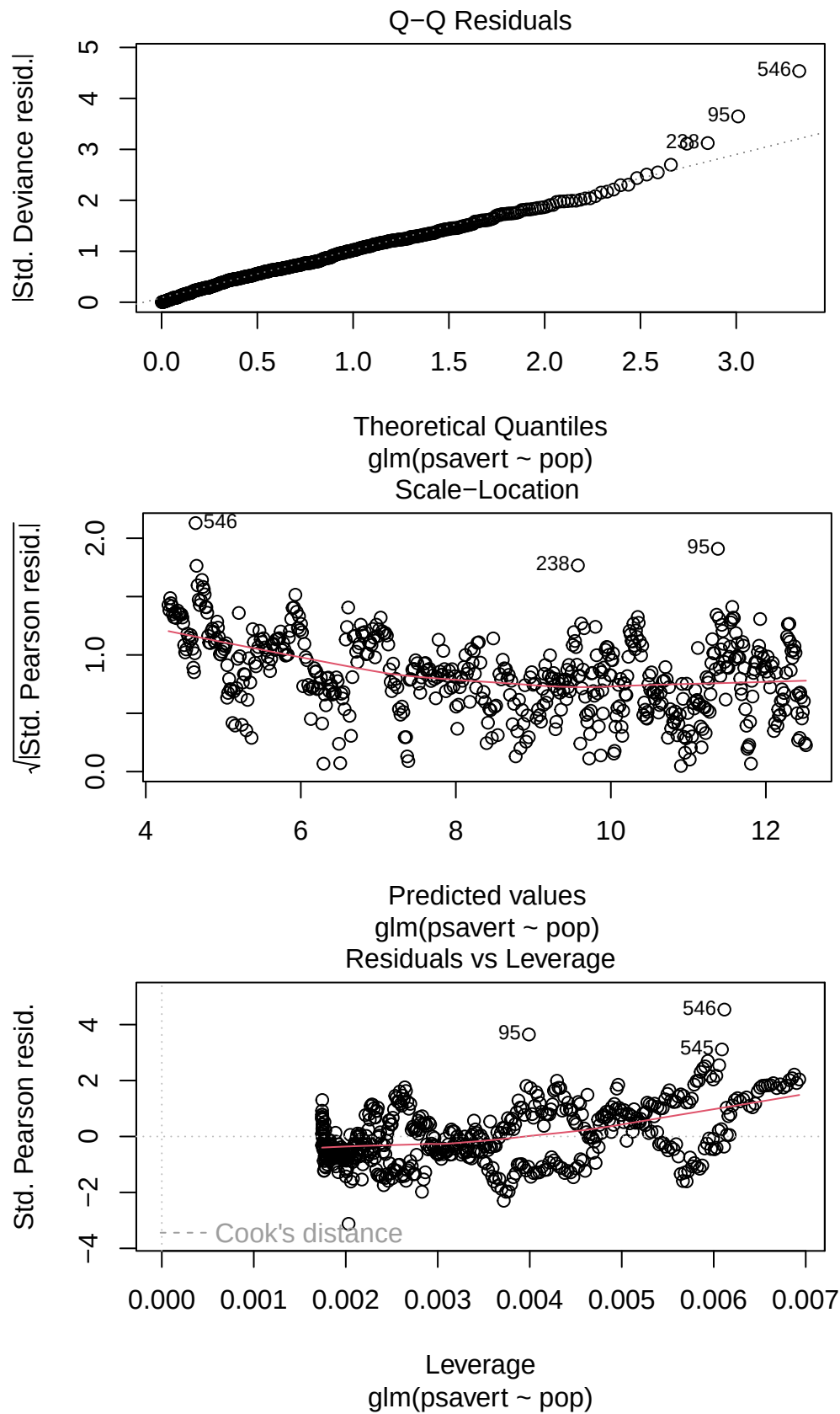
Terms added sequentially (first to last)

	Df	Deviance	Resid. Df	Resid. Dev	F	Pr(>F)
NULL			573	5034.5843		
pop	1	3521.3005	572	1513.2837	1331.0022	< 2.22e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
plot(glm_model1)
```



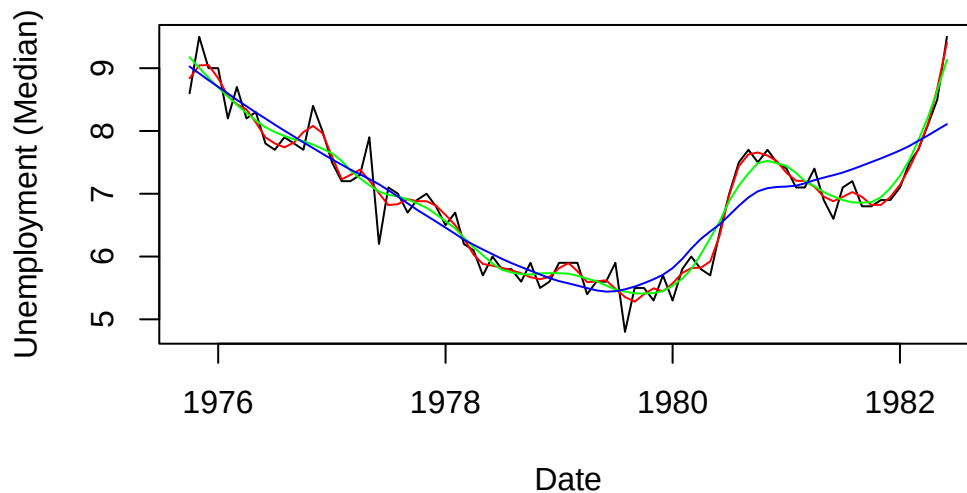


```

economics <- economics[100:180, ] ### narrow span
loessMod10 <- loess(uempmed ~ index, data=economics, span=0.10) # 10% smoothing span
loessMod25 <- loess(uempmed ~ index, data=economics, span=0.25)
loessMod50 <- loess(uempmed ~ index, data=economics, span=0.50)
smoothed10 <- predict(loessMod10)
smoothed25 <- predict(loessMod25)
smoothed50 <- predict(loessMod50)
plot(economics$uempmed, x=economics$date, type="l", main="Loess Smoothing and Prediction", xlab="Date",
     lines(smoothed10, x=economics$date, col="red")
     lines(smoothed25, x=economics$date, col="green")
     lines(smoothed50, x=economics$date, col="blue"))

```

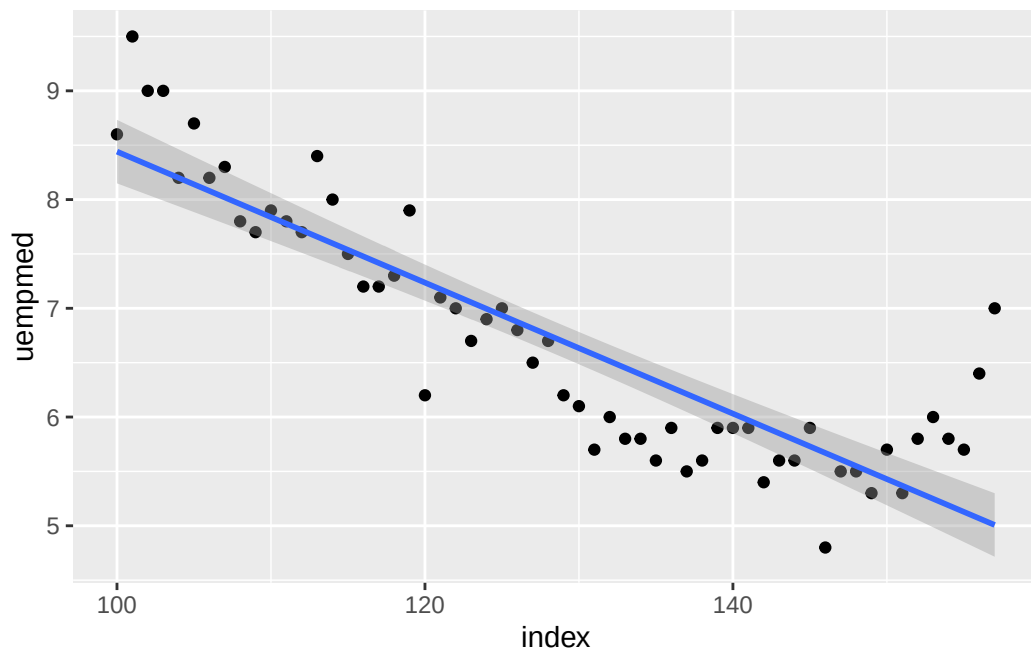
Loess Smoothing and Prediction



```

economics <- economics[1:58,]
library(ggplot2)
ggplot(data=economics, aes(x=index, y=uempmed))+
  geom_point()+
  geom_smooth(method = "lm")

```



I.C.4 One-way ANOVA

```
library(psych)
PlantGrowth ## 내장 dataset
```

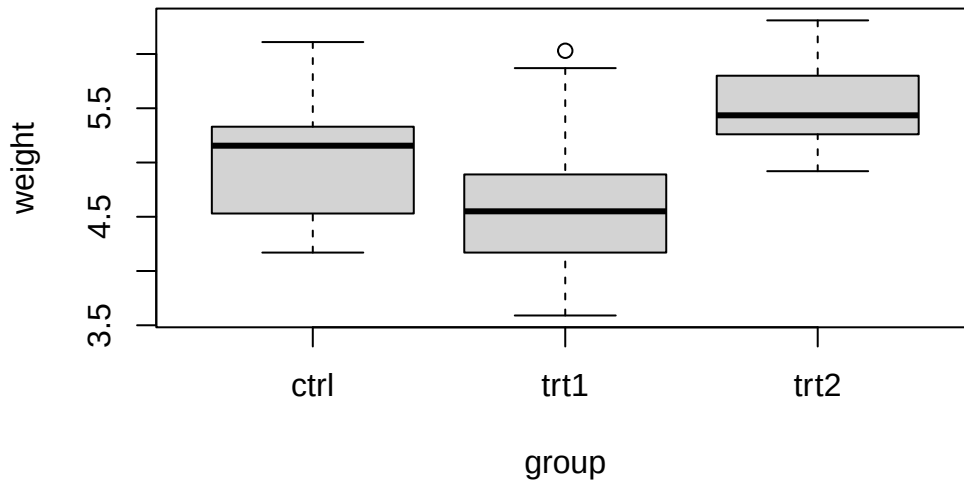
	weight	group
1	4.17	ctrl
2	5.58	ctrl
3	5.18	ctrl
4	6.11	ctrl
5	4.50	ctrl
6	4.61	ctrl
7	5.17	ctrl
8	4.53	ctrl
9	5.33	ctrl
10	5.14	ctrl
11	4.81	trt1
12	4.17	trt1
13	4.41	trt1
14	3.59	trt1
15	5.87	trt1
16	3.83	trt1
17	6.03	trt1
18	4.89	trt1
19	4.32	trt1
20	4.69	trt1
21	6.31	trt2
22	5.12	trt2
23	5.54	trt2
24	5.50	trt2
25	5.37	trt2

```

26  5.29 trt2
27  4.92 trt2
28  6.15 trt2
29  5.80 trt2
30  5.26 trt2

```

```
plot(weight~group, data = PlantGrowth) ## Boxplot으로 자동으로 그려준다.
```



```
with(PlantGrowth, describeBy(weight,group))
```

Descriptive statistics by group

group: ctrl

	vars	n	mean	sd	median	trimmed	mad	min	max	range	skew	kurtosis	se
X1	1	10	5.03	0.58	5.15	5	0.72	4.17	6.11	1.94	0.23	-1.12	0.18

group: trt1

	vars	n	mean	sd	median	trimmed	mad	min	max	range	skew	kurtosis	se
X1	1	10	4.66	0.79	4.55	4.62	0.53	3.59	6.03	2.44	0.47	-1.1	0.25

group: trt2

	vars	n	mean	sd	median	trimmed	mad	min	max	range	skew	kurtosis	se
X1	1	10	5.53	0.44	5.44	5.5	0.36	4.92	6.31	1.39	0.48	-1.16	0.14

```
bartlett.test(PlantGrowth$weight ~ PlantGrowth$group) ## 등분산 가정을 체크함
```

Bartlett test of homogeneity of variances

data: PlantGrowth\$weight by PlantGrowth\$group

Bartlett's K-squared = 2.8785738, df = 2, p-value = 0.2370968

```

aov_model <- aov(weight~group, data = PlantGrowth)
summary(aov_model)

```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
--	----	--------	---------	---------	--------

```
group      2  3.76634 1.8831700 4.84609 0.01591 *
Residuals 27 10.49209 0.3885959
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

I.C.5 Correlation tests

Pearson correlation formula

$$r = \frac{\sum (x - m_x)(y - m_y)}{\sqrt{\sum (x - m_x)^2 \sum (y - m_y)^2}}$$

Spearman correlation formula : non-parametric

$$\rho = \frac{\sum (x' - m_{x'})(y' - m_{y'})}{\sqrt{\sum (x' - m_{x'})^2 \sum (y' - m_{y'})^2}}$$

where $x' = \text{rank}(x)$ and $y' = \text{rank}(y)$

Kendall correlation formula : non-parametric

$$\tau = \frac{n_c - n_d}{\frac{1}{2}n(n-1)}$$

where n_c : number of concordant pairs, n_d : number of discordant pairs, n : size of $x + y$

```
res <- cor.test(economics$index, economics$uempmed, method = "pearson")
res
```

Pearson's product-moment correlation

```
data: economics$index and economics$uempmed
t = -13.654567, df = 56, p-value < 2.2204e-16
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.9255815657 -0.7998071541
sample estimates:
      cor
-0.8769389168
```

```
res$p.value ## res는 리스트형태로 나오는 결과물이다. 여기에서 필요한 값만 골라냄
```

```
[1] 1.822639345e-19
```

```
res$estimate
```

```
cor
-0.8769389168
```

```
res2 <- cor.test(economics$index, economics$uempmed, method = "spearman")
res2
```

Spearman's rank correlation rho

```
data: economics$index and economics$uempmed
S = 60517.721, p-value < 2.2204e-16
alternative hypothesis: true rho is not equal to 0
sample estimates:
rho
-0.861568223
```

```
res3 <- cor.test(economics$index, economics$uempmed, method = "kendall")
res3
```

Kendall's rank correlation tau

```
data: economics$index and economics$uempmed
z = -7.8580733, p-value = 3.90087e-15
alternative hypothesis: true tau is not equal to 0
sample estimates:
tau
-0.7167487038
```

I.C.6 Survival analysis

- rotterdam dataset

A data frame with 2982 observations on the following 15 variables.

pid patient identifier

year year of surgery

age age at surgery

meno menopausal status (0= premenopausal, 1= postmenopausal)

size tumor size, a factor with levels <=20 20-50 >50

grade differentiation grade

nodes number of positive lymph nodes

pgr progesterone receptors (fmol/l)

er estrogen receptors (fmol/l)

hormon hormonal treatment (0=no, 1=yes)

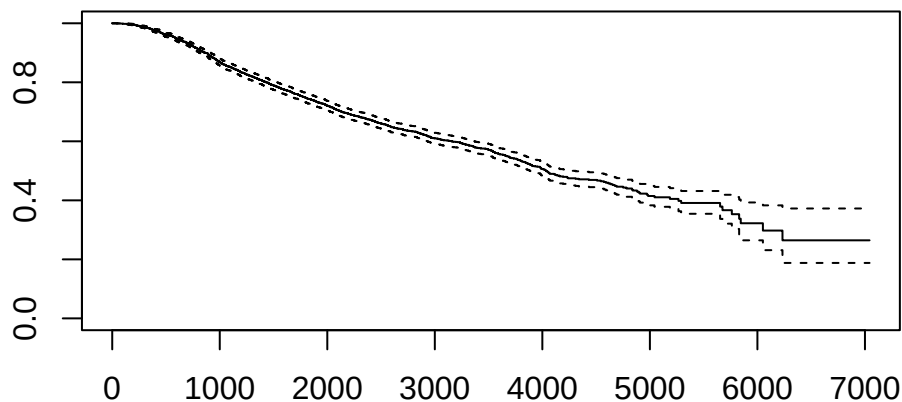
chemo chemotherapy

rtime days to relapse or last follow-up

recur 0= no relapse, 1= relapse
dtime days to death or last follow-up
death 0= alive, 1= dead

- Kaplan Meier Analysis - Basic survival model `survival::Surv`

```
library(rms)
library(survival)
data("cancer")
km <- Surv(rotterdam$dtime, event = rotterdam$death) ## default type : "right"
plot(km) ## km - Surv class (time, status) 가지고 있는 리스트
```



```
median(km); mean(km) ## Surv 객체에 대한 method 함수들이 있다. plot.Surv포함
```

```
$quantile
 50
4033

$lower
 50
3888

$upper
 50
4309

[1] 1302.8833
```

- Kaplan Meier Analysis - `survfit` model

```
km_fit <- survfit(km~rotterdam$meno)
summary(km_fit, c(365*1:19)) ### 정해진 time에 맞는 생존테이블표를 만든다.
```

Call: `survfit(formula = km ~ rotterdam$meno)`

```

      rotterdam$meno=0
time  n.risk  n.event  survival    std.err  lower 95% CI  upper 95% CI
 365    1298     13  0.990084  0.00273656    0.984735    0.995462
 730    1236     56  0.947311  0.00617381    0.935287    0.959489
1095    1140     90  0.878196  0.00905336    0.860630    0.896121
1460    1071     59  0.832587  0.01034896    0.812549    0.853120
1825     973     59  0.786049  0.01140771    0.764005    0.808729
2190     865     50  0.744541  0.01222354    0.720964    0.768888
2555     754     43  0.706203  0.01291841    0.681332    0.731982
2920     611     31  0.674528  0.01353781    0.648509    0.701590
3285     480     15  0.656234  0.01397343    0.629410    0.684201
3650     345     21  0.622823  0.01505424    0.594005    0.653039
4015     217     13  0.594613  0.01631412    0.563482    0.627463
4380     138      6  0.575323  0.01759653    0.541848    0.610866
4745      88      4  0.553799  0.01999709    0.515960    0.594412
5110      54      3  0.530422  0.02334386    0.486587    0.578207
5475      29      2  0.506485  0.02783275    0.454769    0.564082
5840      14      1  0.481161  0.03617160    0.415241    0.557545
6205       5      2  0.390943  0.06657871    0.279996    0.545853
6570       3      0  0.390943  0.06657871    0.279996    0.545853
6935       1      0  0.390943  0.06657871    0.279996    0.545853

```

```

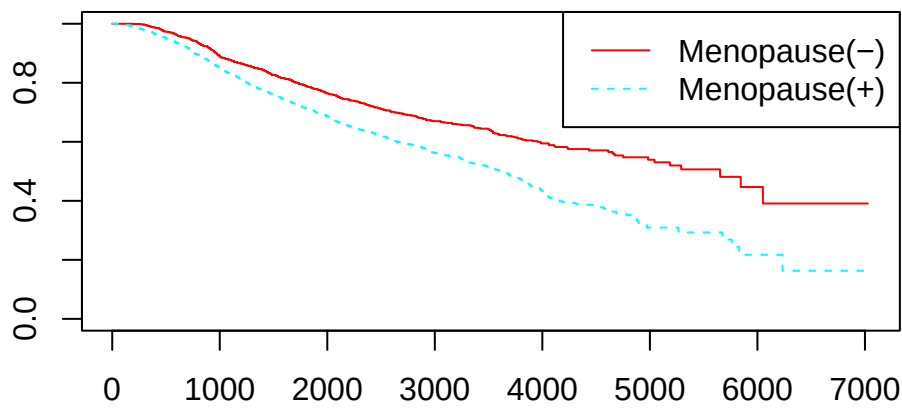
      rotterdam$meno=1
time  n.risk  n.event  survival    std.err  lower 95% CI  upper 95% CI
 365    1616     46  0.972378  0.00401599    0.9645389    0.980281
 730    1508    103  0.910256  0.00701496    0.8966099    0.924109
1095    1366    129  0.832077  0.00918891    0.8142608    0.850283
1460    1245    111  0.764188  0.01045754    0.7439639    0.784962
1825    1111     87  0.709950  0.01121688    0.6883018    0.732278
2190     944     82  0.655456  0.01186297    0.6326122    0.679124
2555     819     58  0.613767  0.01230810    0.5901113    0.638371
2920     642     45  0.577382  0.01272281    0.5529763    0.602864
3285     474     42  0.535877  0.01333692    0.5103646    0.562665
3650     342     31  0.495578  0.01418038    0.4685496    0.524165
4015     188     35  0.430288  0.01613537    0.3997973    0.463104
4380     113     17  0.386353  0.01771621    0.3531444    0.422684
4745      62      6  0.357899  0.01988720    0.3209686    0.399079
5110      28      7  0.309356  0.02431136    0.2651946    0.360871
5475      14      1  0.293074  0.02795732    0.2430961    0.353326
5840       8      3  0.217092  0.04323095    0.1469392    0.320737
6205       4      0  0.217092  0.04323095    0.1469392    0.320737
6570       1      1  0.162819  0.05710016    0.0818823    0.323757
6935       1      0  0.162819  0.05710016    0.0818823    0.323757

```

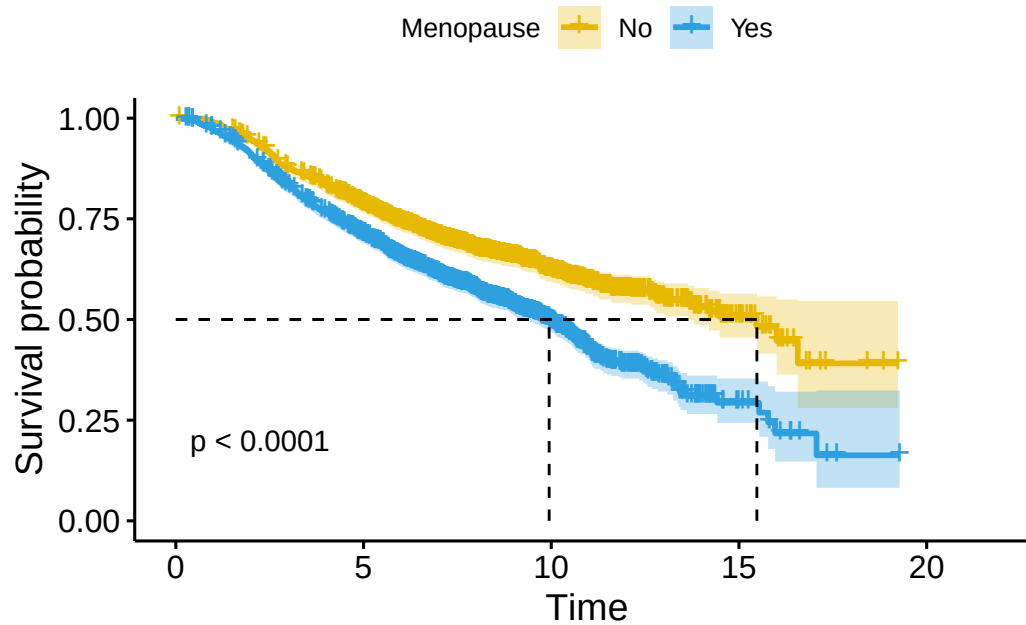
```

plot(km_fit, col = rainbow(2), lty=1:2)
legend("topright", legend = c("Menopause(-)", "Menopause(+)"),
      col= rainbow(2), lty=1:2)

```

```
library(survminer)
ggsurvplot(km_fit, data = rotterdam,
            conf.int = T, xscale = 365.2425, ## xscale can be "d_y"
            break.x.by = 5*365.2425,
            pval = T, pval.size = 4, surv.median.line = "hv",
            risk.table = FALSE, ## if TRUE, risk table is displayed under graph
            legend.title="Menopause", legend.labs=c("No","Yes"),
            palette = c("#E7B800", "#2E9FDF"),)
```



```
## ggplot + survminer package
```

- Cox Proportional model

$$\text{hazard function } h(t) = \lim_{\Delta t \rightarrow 0} \frac{\Pr[(t \leq T < t + \Delta t | T \geq t)]}{\Delta t} = \frac{p(t)}{S(t)}$$

$$\log h_i(t) = \alpha + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_k x_{ik}$$

```
args(coxph)
```

```
function (formula, data, weights, subset, na.action, init, control,
  ties = c("efron", "breslow", "exact"), singular.ok = TRUE,
  robust, model = FALSE, x = FALSE, y = TRUE, tt, method = ties,
  id, cluster, istate, statedata, nocenter = c(-1, 0, 1), ...)
NULL
```

```
library(carData) ## Rossi data set 이용하기 위해서 사용
library(car)      ## Anova function
colnames(Rossi) # emp1-52 : factor (yes or no)
```

```
[1] "week"  "arrest" "fin"    "age"    "race"   "wexp"   "mar"    "paro"
[9] "prio"  "educ"   "emp1"   "emp2"   "emp3"   "emp4"   "emp5"   "emp6"
[17] "emp7"  "emp8"   "emp9"   "emp10"  "emp11"  "emp12"  "emp13"  "emp14"
[25] "emp15" "emp16"  "emp17"  "emp18"  "emp19"  "emp20"  "emp21"  "emp22"
[33] "emp23" "emp24"  "emp25"  "emp26"  "emp27"  "emp28"  "emp29"  "emp30"
[41] "emp31" "emp32"  "emp33"  "emp34"  "emp35"  "emp36"  "emp37"  "emp38"
[49] "emp39" "emp40"  "emp41"  "emp42"  "emp43"  "emp44"  "emp45"  "emp46"
[57] "emp47" "emp48"  "emp49"  "emp50"  "emp51"  "emp52"
```

```
cox_model1 <- coxph(Surv(week, arrest) ~
  fin + age + race + wexp + mar + paro + prio,
  data = Rossi)
summary(cox_model1)
```

Call:

```
coxph(formula = Surv(week, arrest) ~ fin + age + race + wexp +
  mar + paro + prio, data = Rossi)
```

n= 432, number of events= 114

	coef	exp(coef)	se(coef)	z	Pr(> z)
finyes	-0.37942217	0.68425668	0.19137948	-1.98256	0.0474161 *
age	-0.05743774	0.94418067	0.02199947	-2.61087	0.0090312 **
raceother	-0.31389979	0.73059224	0.30799278	-1.01918	0.3081180
wexpyes	-0.14979570	0.86088384	0.21222430	-0.70584	0.4802897
marnot married	0.43370388	1.54296190	0.38186806	1.13574	0.2560642
paroyes	-0.08487108	0.91863070	0.19575667	-0.43355	0.6646124
prio	0.09149708	1.09581358	0.02864855	3.19378	0.0014042 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

	exp(coef)	exp(-coef)	lower .95	upper .95
finyes	0.6842567	1.4614399	0.4702367	0.9956841

```

age          0.9441807  1.0591193  0.9043345  0.9857825
raceother    0.7305922  1.3687526  0.3994948  1.3361001
wexpyes      0.8608838  1.1615969  0.5679354  1.3049390
marnot married 1.5429619  0.6481041  0.7299759  3.2613836
paroyes      0.9186307  1.0885767  0.6259110  1.3482466
prio         1.0958136  0.9125640  1.0359791  1.1591039

```

```

Concordance= 0.64 (se = 0.027 )
Likelihood ratio test= 33.27 on 7 df,  p=2.36e-05
Wald test               = 32.11 on 7 df,  p=3.87e-05
Score (logrank) test = 33.53 on 7 df,  p=2.11e-05

```

```
Anova(cox_model1)
```

Analysis of Deviance Table (Type II tests)

```

      LR Chisq Df Pr(>Chisq)
fin  3.9862101  1  0.0458741 *
age   7.9880173  1  0.0047088 **
race  1.1251518  1  0.2888118
wexp  0.5003372  1  0.4793520
mar   1.4311793  1  0.2315721
paro  0.1869702  1  0.6654503
prio  8.9765972  1  0.0027346 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```
anova(cox_model1)
```

Analysis of Deviance Table

Cox model: response is Surv(week, arrest)
Terms added sequentially (first to last)

```

      loglik      Chisq Df Pr(>|Chi|)
NULL -675.38063
fin  -673.46210  3.83706  1 0.05013146 .
age  -666.23582 14.45257  1 0.00014373 ***
race -665.84148  0.78867  1 0.37450208
wexp -664.21789  3.24717  1 0.07154674 .
mar  -663.57584  1.28411  1 0.25713587
paro -663.23596  0.67976  1 0.40966904
prio -658.74766  8.97660  1 0.00273459 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

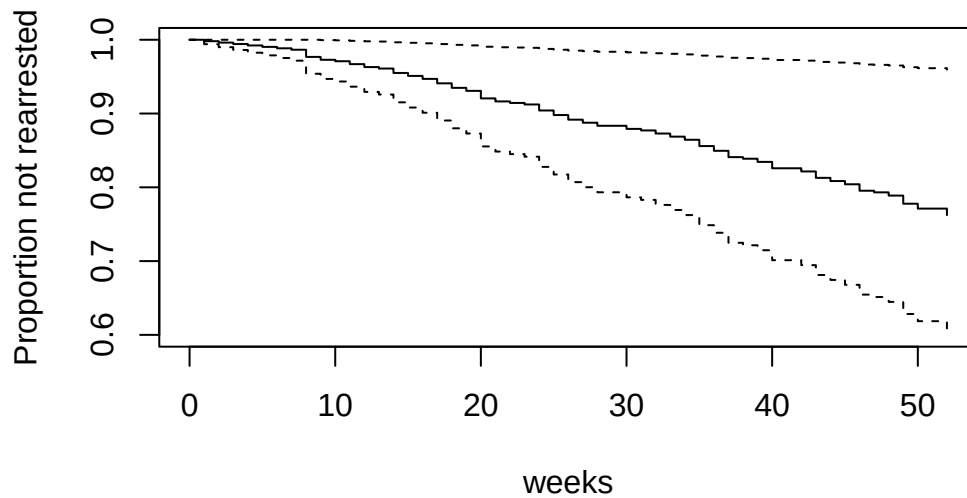
```

모델의 전체적인 생존곡선을 알고 싶으면 survfit 함수를 이용해서 생존곡선을 그릴 수 있다.

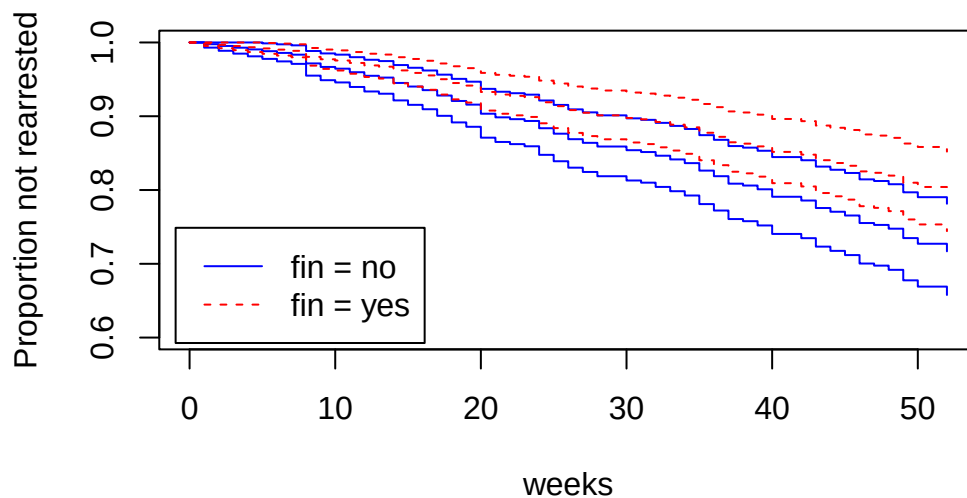
```

plot(survfit(cox_model1), ylim = c(0.6,1),xlab = "weeks",
      ylab = "Proportion not rearrested")

```



```
Rossi.fin <- with(Rossi, data.frame(fin=c(0, 1),
  age=rep(mean(age), 2), race=rep(mean(race == "other"), 2),
  wexp=rep(mean(wexp == "yes"), 2), mar=rep(mean(mar == "not married"), 2),
  paro=rep(mean(paro == "yes"), 2), prio=rep(mean(prio), 2)))
## fin = 0,1 이것을 두그룹으로 나누고 나머지 변수들은 평균적인 값으로 고정해 버림
plot(survfit(cox_model1, newdata = Rossi.fin), conf.int = T,
  lty = c(1,2), ylim = c(0.6,1), xlab = "weeks",
  ylab = "Proportion not rearrested", col = c("blue","red")
)
legend("bottomleft", legend=c("fin = no", "fin = yes"), lty=c(1,2),
  col=c("blue","red") , inset=0.02)
```



I.D 시간과 날짜 변수 다루기 : Management of date-time variables

I.D.1 Date 클래스의 변수 타입

`as.Date(x, format, tryFormats = c("%Y-%m-%d", "%Y/%m/%d"), optional = FALSE, ...)`
 format에 사용되는 코드

code	의미
%d	일(day) : (1부터 31까지의 값) 01-31
%D	시스템 날짜 형식 (mm/dd/yy)
%a	축약형 weekday : (Mon,...)
%A	축약되지 않은 weekday : (Monday,...)
%m	월(month) : (1부터 12까지의 값) 01-12
%b	축약형 월 : (Jan,...)
%B	축약되지 않은 월 : (January,...)
%y	두 자리수 연도 : (19,20,...)
%Y	네 자리수 연도 : (2019,2020,...)

```
rm(list=ls())
library(magrittr)
as.Date("2014-01-24", format = "%Y-%m-%d") -> date01
date01
```

```
[1] "2014-01-24"
```

```
class(date01)
```

```
[1] "Date"
```

```
typeof(date01)
```

```
[1] "double"
```

```
x <- c("May 6 2018", "Aug 20 2019")
as.Date(x, format="%b %d %Y") ## error 발생
```

```
[1] NA NA
```

```
format(Sys.Date(), format = "%b %B %d %D %Y %y %A")
```

```
[1] "3 3월 17 03/17/25 2025 25 월요일"
```

```
x <- c("5 6 2018", "8 20 2019")
as.Date(x, format="%b %d %Y")
```

```
[1] "2018-05-06" "2019-08-20"
```

```
x <- c("20180506", "20190820")
as.Date(x, format="%Y%m%d")
```

```
[1] "2018-05-06" "2019-08-20"
```

```
date02 <- as.Date(x, format="%Y%m%d")
date02[,2] - date02[,1] ## error
```

Error in `[.default` (date02, , 2): incorrect number of dimensions

```
diffdays <- diff(date02, units = "days")
diffdays
```

Time difference of 471 days

```
class(diffdays)
```

```
[1] "difftime"
```

```
str(diffdays)
```

```
'difftime' num 471
- attr(*, "units")= chr "days"
```

```
as.numeric(diffdays)
```

```
[1] 471
```

```
diff(date02, units = "month") ## error
```

Error in match.arg(dots\$units, choices = setdiff(eval(formals(difftime)\$units), : 'arg' should be one

```
##units should be one of "secs", "mins", "hours", "days", "weeks"
as.numeric(diffdays)/365.24219 *12
```

```
[1] 15.47466354
```

I.D.2 POSIX class

POSIXlt, POSIXct

뒤에 있는 "lt"는 list time, "ct"는 continuous time이고 정리하면 POSIX + lt -> POSIXlt, POSIX + ct -> POSIXct

```
time.list <- as.POSIXlt(Sys.time())
time.list
```

```
[1] "2025-03-17 08:56:11 KST"
```

```
unlist(time.list)
```

	sec	min	hour	mday
	"11.7140250205994"	"56"	"8"	"17"
	mon	year	wday	yday
	"2"	"125"	"1"	"75"
	isdst	zone	gmtoff	
	"0"	"KST"	"32400"	

```
unlist(time.list)%>%t%>%t
```

```
[,1]
sec   "11.7140250205994"
min   "56"
hour  "8"
mday  "17"
mon   "2"
year  "125"
wday  "1"
yday  "75"
isdst "0"
zone  "KST"
gmtoff "32400"
```

```
class(time.list)
```

```
[1] "POSIXlt" "POSIXt"
```

```
str(time.list)
```

```
POSIXlt[1:1], format: "2025-03-17 08:56:11"
```

```
time.cont <- as.POSIXct(Sys.time())
time.cont
```

```
[1] "2025-03-17 08:56:11 KST"
```

```
class(time.cont)
```

```
[1] "POSIXct" "POSIXt"
```

```
str(time.cont)
```

```
POSIXct[1:1], format: "2025-03-17 08:56:11"
```

POSIX.lt 내용을 살펴보면,

- sec : 그냥 초

- min : 그냥 분
- hour : 그냥 시간
- mday: 1일을 1로
- mon: 1월을 0으로
- year : 1900년을 0으로
- wday: 일요일이 0으로
- yday : 1월 1일이 0으로
- isdst : 서머타임

```
diff(c(time.list, Sys.time() ))
```

Time difference of 0.03303194046 secs

```
## units should be one of "secs", "mins", "hours", "days", "weeks"
```

원래의 시간에 시간을 더하거나 빼는 것은 어떻게 하나?

R에서는 시간이 전부 초로 계산됨

```
Sys.time() + 3600*24*15 ## 오늘 날짜에 15일을 더함
```

```
[1] "2025-04-01 08:56:11 KST"
```

```
library(lubridate)
Sys.time() + days(15)
```

```
[1] "2025-04-01 08:56:11 KST"
```


I.D.3 Lubridate package

lubridate라는 패키지를 사용하면 시간 연산이 편리함 "period" class 사용

Order of elements in date-time	Parse function
year, month, day	ymd()
year, day, month	ydm()
month, day, year	mdy()
day, month, year	dmy()
hour, minute	hm()
hour, minute, second	hms()
year, month, day, hour, minute, second	ymd_hms()

```
library(lubridate)
ymd("20110604")
```

```
[1] "2011-06-04"
```

```
mdy("06-04-2011")
```

```
[1] "2011-06-04"
```

```
dmy("04/06/2011")
```

```
[1] "2011-06-04"
```

```
ymd(20240105)
```

```
[1] "2024-01-05"
```

```
#### period 는 연산이 가능함
period(second = 3, minute = 1, hour = 2, day = 13, week = 1)-days(1)
```

```
[1] "19d 2H 1M 3S"
```

```
dmy("04/06/2011") + period(second = 3, minute = 1, hour = 2, day = 13, week = 1)
```

```
[1] "2011-06-24 02:01:03 UTC"
```

```
#### _h, _hm, _hms 등을 붙이면 시간:분:초 까지 시간을 만들 수 있다.
arrive <- ymd_hms("2011-06-04 12:00:00", tz = Sys.timezone(location = TRUE))
arrive
```

```
[1] "2011-06-04 12:00:00 KST"
```

```
wday(arrive)
```

```
[1] 7
```

```
wday(arrive, label = TRUE)
```

```
[1] 토
```

```
Levels: 일 < 월 < 화 < 수 < 목 < 금 < 토
```

```
arrive %>% class
```

```
[1] "POSIXct" "POSIXt"
```

```
ymd_hms("2011-06-04 12:00:00") ### time zone을 설정하지 않으면 오류가 생길 수 있다
```

```
[1] "2011-06-04 12:00:00 UTC"
```

```
### "UTC" = GMT Coordinated Universal Time
Sys.timezone(location = TRUE) # 현재 사용하는 시스템의 time zone
```

```
[1] "Asia/Seoul"
```

```
#### OlsonNames() : 사용가능한 모든 time zone의 이름들
now("GMT")
```

```
[1] "2025-03-16 23:56:12 GMT"
```

```
leave <- ymd_hms("2024-11-30 18:30:00", tz="Asia/Seoul")
arrive <- ymd_hms("2024-11-30 12:20:00", tz="US/Pacific")
difftime(arrive,leave)
```

Time difference of 10.83333333 hours

```
### 시간 더하기
now() + minutes(5)
```

```
[1] "2025-03-17 09:01:12 KST"
```

```
now() + months(-5)
```

```
[1] "2024-10-17 08:56:12 KST"
```

```
now() + years(5)
```

```
[1] "2030-03-17 08:56:12 KST"
```

```
jan31 <- ymd("2013-01-31")
jan31 + months(0:11)
```

```
[1] "2013-01-31" NA          "2013-03-31" NA          "2013-05-31"
[6] NA          "2013-07-31" "2013-08-31" NA          "2013-10-31"
[11] NA          "2013-12-31"
```

Date component	Accessor
Year	year()
Month	month()
Week	week()
Day of year	yday()
Day of month	mday()
Day of week	wday()
Hour	hour()
Minute	minute()
Second	second()
Time zone	tz()

```
library(readxl)
dir(pattern = "*.xlsx")
```

```
[1] "datasummary.xlsx"      "operation.xlsx"        "patients.xlsx"
[4] "폐암-항암.xlsx"       "폐암 -환자정보.xlsx"  "폐암 op.xlsx"
[7] "폐암 RT.xlsx"         "폐암환자 통계.xlsx"
```

```
opdata <- read_xlsx(dir(pattern = "*.xlsx")[2], sheet = 2)
ptdata <- read_xlsx(dir(pattern = "*.xlsx")[3], sheet = 1)
str(opdata)
```

```
tibble [2,657 x 2] (S3: tbl_df/tbl/data.frame)
 $ 등록정보: chr [1:2657] "00679325" "00249356" "00251488" "00250179" ...
 $ 수술일   : chr [1:2657] "1993-08-19" "1982-06-02" "1982-06-22" "1982-06-08" ...
```

```
str(ptdata)
```

```
tibble [16,921 x 5] (S3: tbl_df/tbl/data.frame)
 $ 등록번호   : chr [1:16921] "00000000" "00000001" "00000039" "00000103" ...
 $ 성별       : chr [1:16921] "M" "F" "F" "M" ...
 $ 나이       : chr [1:16921] NA "48" "54" "-29" ...
 $ 진단일     : chr [1:16921] NA "1963-12-26" "1963-11-14" "1963-06-01" ...
 $ 최종추적일: chr [1:16921] NA "1999-07-29" "1964-11-14" "1964-06-01" ...
```

```
opdf <- merge(ptdata,opdata,by.x="등록번호", by.y="등록정보", all=FALSE )
str(opdf)
```

```
'data.frame': 2654 obs. of 6 variables:
 $ 등록번호 : chr "00000636" "00001671" "00002686" "00002793" ...
 $ 성별 : chr "M" "M" "M" "M" ...
 $ 나이 : chr "39" "48" "41" "48" ...
 $ 진단일 : chr "1964-05-19" "1964-02-04" "1965-09-29" "1965-08-23" ...
 $ 최종추적일: chr "1964-05-19" "1965-10-11" "1966-09-29" "1966-08-23" ...
 $ 수술일 : chr "1964-05-22" "1964-02-04" "1965-09-21" "1965-08-23" ...
```

```
colnames(opdf) <- c("chartno","sex","age","dxdate", "fudate", "opdate")
```

```
opdf2 <- opdf
colnames(opdf2)
```

```
[1] "chartno" "sex" "age" "dxdate" "fudate" "opdate"
```

```
date.varname <- grep( "date",colnames(opdf2) , value=T )
date.varname
```

```
[1] "dxdate" "fudate" "opdate"
```

```
for (c in date.varname) {
  opdf2[,c] <- as.POSIXct(opdf2[,c], format="%Y-%m-%d",
                          tz= "Asia/Seoul" )
}
str(opdf2)
```

```
'data.frame': 2654 obs. of 6 variables:
 $ chartno: chr "00000636" "00001671" "00002686" "00002793" ...
 $ sex : chr "M" "M" "M" "M" ...
 $ age : chr "39" "48" "41" "48" ...
 $ dxdate : POSIXct, format: "1964-05-19" "1964-02-04" ...
 $ fudate : POSIXct, format: "1964-05-19" "1965-10-11" ...
 $ opdate : POSIXct, format: "1964-05-22" "1964-02-04" ...
```

```
head(opdf2$fudate)
```

```
[1] "1964-05-19 KST" "1965-10-11 KST" "1966-09-29 KST" "1966-08-23 KST"
[5] "1965-12-27 KST" "1967-01-25 KST"
```

```
opdf2$surv.period <- opdf2$fudate - opdf2$opdate
str(opdf2$surv.period)
```

```
'difftime' num [1:2654] -259200 53136000 32227200 31536000 ...
- attr(*, "units")= chr "secs"
```

```
opdf2$urv.period <- difftime(opdf2$fudate, opdf2$opdate, units = "days")
str(opdf2$urv.period)
```

```
'difftime' num [1:2654] -3 615 373 365 ...
- attr(*, "units")= chr "days"
```

```
opdf2$urvival.month <- opdf2$urv.period /365.24219 *12
attr(opdf2$urvival.month, "units") <- "months"
str(opdf2)
```

```
'data.frame': 2654 obs. of 8 variables:
 $ chartno : chr "00000636" "00001671" "00002686" "00002793" ...
 $ sex : chr "M" "M" "M" "M" ...
 $ age : chr "39" "48" "41" "48" ...
 $ dxdate : POSIXct, format: "1964-05-19" "1964-02-04" ...
 $ fudate : POSIXct, format: "1964-05-19" "1965-10-11" ...
 $ opdate : POSIXct, format: "1964-05-22" "1964-02-04" ...
 $ urv.period : 'difftime' num -3 615 373 365 ...
 ..- attr(*, "units")= chr "days"
 $ urvival.month: 'difftime' num -0.0985647359085214 20.2057708612469 12.2548821646262 11.9920428688
 ..- attr(*, "units")= chr "months"
```

I.D.4 ID 주민번호에서 생일 추출하기

주민번호는 연도, 월, 일 두자리씩 + 1또는2는 1900년대 남녀, 3또는4는 2000년대 남녀 구분

Regular expression으로 정상적인 ID 인지 체크하고 날짜 형식으로 변형

```
library(lubridate)
library(stringr)
IDno <- "120923-3012123"
patternid <- "(\\d{2})([01]\\d)(([0-2]\\d)|(3[01]))-([1-4])"
match_id <- str_match(IDno, patternid)
match_id
```

```
 [,1] [,2] [,3] [,4] [,5] [,6] [,7]
[1,] "120923-3" "12" "09" "23" "23" NA "3"
```

```
ymd_str <- paste(match_id[,2:4], collapse = " ")
c_str <- ifelse(match_id[,7]<"3","19","20")
as.Date(paste(c_str,ymd_str,sep = "-"), format = "%Y%m%d", tz= "Asia/Seoul" )
```

```
[1] "2012-09-23"
```

I.D.5 PFT report에서 검사날짜와 검사치 추출하기

```
library(tesseract)
setwd(paste(getwd(), "/TMP", sep=""))
setwd("G:/R project/nomogram/TMP")
jpgfiles <- dir(pattern = ".jpg")
jpgfiles
```

```
[1] "00872290_20120803_2001_F6001_001.jpg"
[2] "00872290_20150511_2024_F60021_001.jpg"
[3] "00872290_20170324_2022_F60021_001.jpg"
[4] "00872290_20200519_2019_F60021_001.jpg"
[5] "00872290_20210601_2016_F60021_001.jpg"
[6] "00872290_20220329_2004_F6002_001.jpg"
[7] "00872290_20241121_2014_F60021_001.jpg"
[8] "01037742_20100810_2001_F6001_001.jpg"
[9] "01037742_20110512_2015_F6001_001.jpg"
[10] "01043414_20240926_2014_F60021_001.jpg"
[11] "01220463_20180626_2010_F6002_001.jpg"
[12] "01220463_20200326_2032_F60021_001.jpg"
[13] "01220463_20210506_2006_F60021_001.jpg"
[14] "01220463_20230615_2003_F60021_001.jpg"
[15] "01320646_20241118_2007_F60021_001.jpg"
[16] "01328776_20241115_2002_F60021_001.jpg"
[17] "01345136_20241031_2013_F6010_001.jpg"
[18] "01345136_20241031_2014_F60021_001.jpg"
[19] "01367827_20241101_2014_F60021_001.jpg"
```

```
pfttext <- NULL
for (files in jpgfiles) {
  pfttext <- c(pfttext, ocr(files, engine = tesseract("kor+eng")))
}
setwd("G:/R project/nomogram")
str(pfttext)
```

```
chr [1:19] "Name: 이 금 주 , 10: 00872290\nGender: Female Room: C7 Date: 08/03/12\nAge: 51 Race: Asia"
```

```
pattern1 <- "(\\d{2}/\\d{2}/\\d{2})|(\\d{2,4}[ -]\\d{2}[ -]\\d{2})"
match_date0 <- str_match(pfttext, pattern1)
match_date0
```

```
      [,1]      [,2]      [,3]
[1,] "08/03/12" "08/03/12" NA
[2,] "05/11/15" "05/11/15" NA
[3,] "03/27/17" "03/27/17" NA
[4,] "05/19/20" "05/19/20" NA
[5,] "06/01/21" "06/01/21" NA
```

```
[6,] "03/29/22" "03/29/22" NA
[7,] "2024-11-21" NA "2024-11-21"
[8,] "2010 08 10" NA "2010 08 10"
[9,] "2011 05 12" NA "2011 05 12"
[10,] "2024-10-29" NA "2024-10-29"
[11,] "06/26/18" "06/26/18" NA
[12,] "03/27/20" "03/27/20" NA
[13,] "08/12/21" "08/12/21" NA
[14,] "2023-12-21" NA "2023-12-21"
[15,] "2024-12-05" NA "2024-12-05"
[16,] "2024-11-15" NA "2024-11-15"
[17,] "2024-11-06" NA "2024-11-06"
[18,] "2024-11-06" NA "2024-11-06"
[19,] "2024-11-01" NA "2024-11-01"
```

```
match_date1 <- as.POSIXct(match_date0[,1], tz="Asia/Seoul", "%Y-%m-%d")
match_date2 <- as.POSIXct(match_date0[,1], tz="Asia/Seoul", "%Y %m %d")
match_date3 <- as.POSIXct(match_date0[,1], tz="Asia/Seoul", "%m/%d/%y")
match_date <- cbind(match_date1, match_date2, match_date3)
match_date <- apply(match_date, MARGIN = 1, function(x) {sum(x, na.rm = TRUE)})
match_date <- as.POSIXct(match_date)
match_date
```

```
[1] "2012-08-03 KST" "2015-05-11 KST" "2017-03-27 KST" "2020-05-19 KST"
[5] "2021-06-01 KST" "2022-03-29 KST" "2024-11-21 KST" "2010-08-10 KST"
[9] "2011-05-12 KST" "2024-10-29 KST" "2018-06-26 KST" "2020-03-27 KST"
[13] "2021-08-12 KST" "2023-12-21 KST" "2024-12-05 KST" "2024-11-15 KST"
[17] "2024-11-06 KST" "2024-11-06 KST" "2024-11-01 KST"
```

```
pattern_fev1 <- "FEV1[ ]+[\\w\\W]{3,6}[ ]+\\d?[\\d.]\\d{0,2}[ ]+(\\d?[\\d.]\\d{0,2})[ ]+(\\d?[\\d.]\\d{0,2})[ ]+(\\d?[\\d.]\\d{0,2})"
str_match(pfttext, pattern_fev1)
```

```
      [,1]      [,2]      [,3]
[1,] "FEV1 Liters 2.40 2.17 90" "2.17" "90"
[2,] "FEV1 Liters 2.29 253 110" "253" "110"
[3,] "FEV1 Liters 2.23 2.39 107" "2.39" "107"
[4,] "FEV1 Liters 2.13 2.07 97" "2.07" "97"
[5,] "FEV1 Liters 2.14 1.96 92" "1.96" "92"
[6,] "FEV1 Liters 2.10 1.97 94" "1.97" "94"
[7,] "FEV1 [L] 2.06 1.92 93" "1.92" "93"
[8,] "FEV1 Liters 251 3.16 126" "3.16" "126"
[9,] "FEV1 Liters 251 293 116" "293" "116"
[10,] "FEV1 [L] 1.69 1.97 116" "1.97" "116"
[11,] "FEV1 Liters 2.79 3.12 112" "3.12" "112"
[12,] "FEV1 Liters 2.65 2.98 113" "2.98" "113"
[13,] "FEV1 Liters 2.59 2.76 107" "2.76" "107"
[14,] "FEV1 [L] 2.53 2.79 111" "2.79" "111"
[15,] "FEV1 [L] 2.65 2.17 82" "2.17" "82"
```

```
[16,] "FEV1 [L] 1.43 0.85 59"      "0.85" "59"
[17,] NA                          NA      NA
[18,] "FEV1 [L] 2.89 2.11 73"     "2.11" "73"
[19,] "FEV1 [L] 2.34 2.80 120"    "2.80" "120"
```

```
match_fev1 <- str_match(pfttext, pattern_fev1)[,2]
match_fev1p <- str_match(pfttext, pattern_fev1)[,3]
pattern_chartno <- "\\d{8}"
match_chartno <- str_match(pfttext, pattern_chartno)
df <- data.frame(chartno=match_chartno, date = match_date, fev1 = match_fev1, fev1p = match_fev1p)
df$fev1 <- as.numeric(df$fev1)
df$fev1p <- as.numeric(df$fev1p)
df$fev1[df$fev1>10 & !is.na(df$fev1)] <- df$fev1[df$fev1>10 & !is.na(df$fev1)]/100
df
```

```
      chartno      date fev1 fev1p
1 00872290 2012-08-03 2.17    90
2 00872290 2015-05-11 2.53   110
3 00872290 2017-03-27 2.39   107
4 00872290 2020-05-19 2.07    97
5 00872290 2021-06-01 1.96    92
6 00872290 2022-03-29 1.97    94
7 00872290 2024-11-21 1.92    93
8 01037742 2010-08-10 3.16   126
9 01037742 2011-05-12 2.93   116
10 01043414 2024-10-29 1.97   116
11 01220463 2018-06-26 3.12   112
12 01220463 2020-03-27 2.98   113
13 01220463 2021-08-12 2.76   107
14 01220463 2023-12-21 2.79   111
15 01320646 2024-12-05 2.17    82
16 01328776 2024-11-15 0.85    59
17 01345136 2024-11-06  NA     NA
18 01345136 2024-11-06 2.11    73
19 01367827 2024-11-01 2.80   120
```

I.D.6 Interval class

lubridate package에 있는 함수

```
interval_fu <- interval(opdf2$opdate, opdf2$fudate)
(test_interval <- interval_fu[1000])
```

```
[1] 1997-08-13 KST--2000-09-14 KST
```

```
int_start(test_interval)
```

```
[1] "1997-08-13 KST"
```



```
int_end(test_interval)
```

```
[1] "2000-09-14 KST"
```

```
int <- interval(ymd("2001-01-01"), ymd("2002-01-01"))
int_length(int)
```

```
[1] 31536000
```

```
int <- interval(ymd("2001-01-01"), ymd("2002-01-01"))
int_flip(int)
```

```
[1] 2002-01-01 UTC--2001-01-01 UTC
```

```
int <- interval(ymd("2001-01-01"), ymd("2002-01-01"))
int_shift(int, duration(days = 11))
```

```
[1] 2001-01-12 UTC--2002-01-12 UTC
```

```
int_shift(int, duration(hours = -1))
```

```
[1] 2000-12-31 23:00:00 UTC--2001-12-31 23:00:00 UTC
```

```
int1 <- interval(ymd("2001-01-01"), ymd("2002-01-01"))
int2 <- interval(ymd("2001-06-01"), ymd("2002-06-01"))
int3 <- interval(ymd("2003-01-01"), ymd("2004-01-01"))

int_overlaps(int1, int2) # TRUE
```

```
[1] TRUE
```

```
int_overlaps(int1, int3) # FALSE
```

```
[1] FALSE
```

II Advanced Techniques

II.A Data Manipulation

II.A.1 Data reading

data file이 존재하는 디렉토리를 먼저 설정해주어야 한다. 이를 위한 명령어는 `setwd() = set working directory` 라는 의미 `setwd("G:/R project/nomogram")` 와 같이 디렉토리를 설정해줄 수도 있지만,

만약 디렉토리를 찾기 어렵다면 `setwd(choose.dir())` 와 같은 명령으로 파일탐색기를 열어서 디렉토리를 선택할 수 있다. 현재 사용할 `xlsx` 파일들이 다음 디렉토리에 있다고 가정하자

```
setwd("G:/R project/nomogram")
library(readxl)
dir(pattern = "*.xlsx")
```

```
[1] "datasummary.xlsx"      "operation.xlsx"      "Patient_info.xlsx"
[4] "patients.xlsx"        "Survdata.xls"       "폐암-항암.xlsx"
[7] "폐암 -환자정보.xlsx"  "폐암 op.xlsx"       "폐암 RT.xlsx"
[10] "폐암_OP.xls"          "폐암_RT.xls"        "폐암_추적조사.xlsx"
[13] "폐암_항암치료.xls"    "폐암환자 통계.xlsx"
```

```
xlsxfiles <- dir(pattern = "*.xlsx")
ptinfo <- read_xls(xlsxfiles[5])
```

II.A.2 Binding tables

데이터프레임 결합 방법들

`rbind()`, `cbind()`, `merge()`

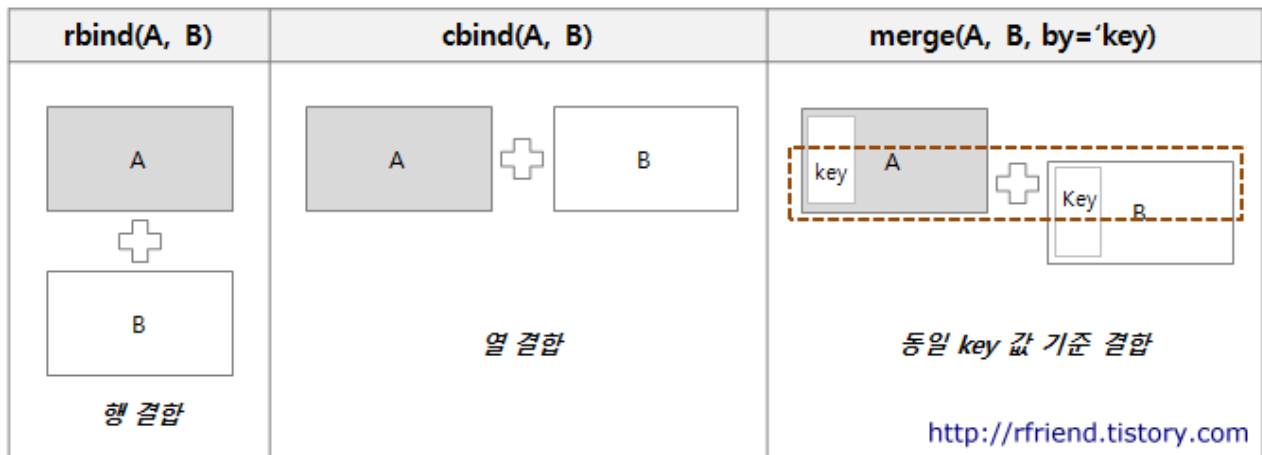


Figure 1: 데이터프레임 결합방법

** 당연한 이야기지만 `rbind`는 컬럼의 갯수가 같아야 하고, `cbind`는 행의 갯수가 같아야 함

II.A.3 Join (Merge) tables

merge function

```
merge(x, y, by = intersect(names(x), names(y)), ## 공통된 컬럼하나를 결합용 키로 선택
      by.x = by, by.y = by, all = FALSE, all.x = all, all.y = all, ## x와 y의 결합용 키의 이름이 서로 다를 경우에는
      독립적으로 지정
      sort = TRUE, suffixes = c(".x", ".y"), no.dups = TRUE,
      incomparables = NULL, ...)
```

```
df1 <- data.frame( ID = 1:10, Name = c("Lee","Kim","Park", "Kang",
                                       "Shin", "Lim", "Kwon", "Choi", "Nam", "Baek" ),
                  Score = as.integer(rnorm(10, 80,6 ))
                )
df2 <- data.frame( ID = sample(1:10, 9, replace = F),
                  Department = sample( c("IM","GS","GY","PD" ),9, replace = T),
                  Age = as.integer(rnorm(9, 40,6 )) )

df1
```

	ID	Name	Score
1	1	Lee	73
2	2	Kim	79
3	3	Park	83
4	4	Kang	88
5	5	Shin	83
6	6	Lim	72
7	7	Kwon	82
8	8	Choi	77
9	9	Nam	76
10	10	Baek	77

```
df2
```

	ID	Department	Age
1	10	IM	50
2	8	GY	39
3	2	PD	51
4	7	GY	43
5	9	GS	44
6	3	GY	49
7	1	IM	32
8	6	GS	46
9	4	GY	47

```
merged_df <- merge(df1,df2, by="ID", all = TRUE) # full join
merged_df
```

	ID	Name	Score	Department	Age
--	----	------	-------	------------	-----

1	1	Lee	73	IM	32
2	2	Kim	79	PD	51
3	3	Park	83	GY	49
4	4	Kang	88	GY	47
5	5	Shin	83	<NA>	NA
6	6	Lim	72	GS	46
7	7	Kwon	82	GY	43
8	8	Choi	77	GY	39
9	9	Nam	76	GS	44
10	10	Baek	77	IM	50

II.A.4 Types of Join

merge 함수를 실행하여 데이터를 결합할 때에는 데이터 join 방법이 다음과 같이 4가지가 있다.

두개의 df에서 모든 데이터가 완전하게 존재하지 않기 때문에 일치하지 않는 부분에 대한 처리규칙이 중요하다.

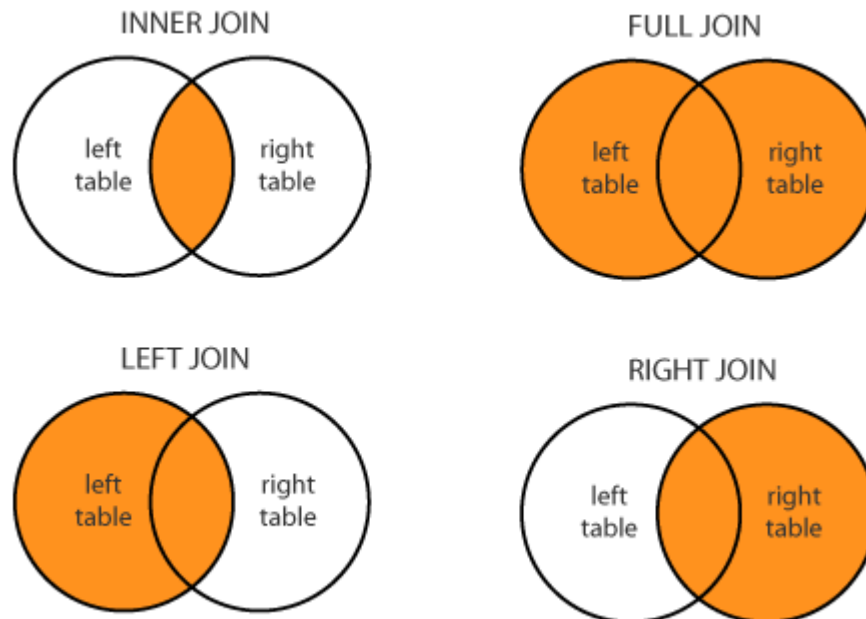


Figure 2: Types of Join

merge 함수의 옵션에서 all = TRUE 를 선택하면 full join, all.x 는 left join, all.y는 right join이 된다.

all= FALSE 인 경우에는 당연히 inner join

dplyr package에는 개별적인 join 함수가 있는데 그것을 사용해도 됨

inner_join(df1, df2), left_join(df1, df2), right_join(df1, df2), full_join(df1, df2)

left_join(df2, df1) : alternative right join

II.A.5 Reshape data

II.B Pipeline operator

library magrittr 를 사용하면 pipeline 연산자를 쓸 수 있게 된다. %>% 형식이다.

만약 c("A","B","C") 라는 데이터를 "ABC" 로 paste 한 다음에 다시 tolower 함수를 적용하여 "abc"로 변환하는 작업을 한다고 하자. 그런 경우에는 다음과 같이 코딩을 해야 한다. 하지만 pipeline operator를 사용하면 함수 중첩을 줄이고 코드를 이해하기 쉽게 사용할 수 있다.

```
library(magrittr)
tolower(paste(c("A","B","C"), collapse = ""))
```

```
[1] "abc"
```

```
c("A","B","C") %>% paste(., collapse = "") %>% tolower ## paste 함수에는 여러 인자가 들어가는데
```

```
[1] "abc"
```

```
## 첫번째 인자로 들어가기 위해서 . 을 사용함
```

II.C Regular Expression

정규표현식의 정의 : 특정 문자열을 가리키는 패턴

아주 간단하고 쉬운 정규표현식의 한가지 예를 들어보면

`\d`

이것은 0부터 9까지의 숫자와 매칭된다. 그리고 다음처럼 제법 복잡하게 발전시킬 수도 있다.

`^0\d{1,2}-?\d{2,4}-?\d{4}`

이 패턴은 한국의 전화번호에 대응하는 패턴인데 0으로 시작하는 핸드폰번호 또는 지역번호 - 0으로 시작하지 않는 2~4자리 번호 - 4자리 번호와 매칭이 되는 패턴이다.

II.C.1 *RegexPal* 연습하기

<http://www.regexpal.com>

이 웹사이트에 들어가면 위에 정규표현식 입력영역이 있고 아래에 목표 텍스트 영역이 있다.

<https://regexr.com/> : 설명이 같이 있어서 복잡한 패턴을 공부하는데 도움이 된다.

일단 *RegexPal*로 숫자 패턴 연습을 해보자. 아래 영역에는 다양한 전화번호 또는 여러가지 번호나 텍스트를 붙여넣어놓자 위의 영역을 지우고 3이라는 숫자만 입력해보자

`[0-9]` 라고 입력해보자

숫자의 범위를 제한해보자 0,1,2,3,5,8 의 번호만 찾고 싶으면 `[012358]` 을 입력한다

11개의 숫자를 찾는 방법은? `[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]`

`[0-9]` 표현은 다음과 같은 단축문자로 바꿀 수 있다.

`\d`

다음과 같이 전화번호를 찾아볼 수 있다

`\d\d\d-\d\d\d-\d\d\d\d`

숫자 이외의 문자를 찾는데 쓰는 단축문자를 쓰면..

`\d\d\dD\d\d\dD\d\d\d\d`

`\d\d\d.\d\d\d\d.\d\d\d\d`

점 (.) 을 사용하면 대부분의 아무 문자나 찾을 수 있는 와일드카드 역할을 한다. (개행문자 제외)

수량자를 사용해보자

`\d{3}-?\d{3}-?\d{4}`

{3} 중괄호 안의 숫자는 몇번 나오는지 숫자를 지정한다. {3,9}는 3~9회, {2,}는 2회 이상, {,99}는 99회 이하, ..

물음표(?) 도 수량자인데 한개 이하를 의미 (0 또는 1)

덧셈기호(+)는 하나 이상

별표(*)는 0 이상을 의미

`(\d{3,4}[-.]?)+`

위의 정규식은 숫자 세개 또는 네개 다음에 점이나 하이픈이 한번 이하로 등장하는 패턴을 묶은 그룹이 한번 이상 등장한다는 뜻이다. (괄호는 참조그룹의 의미)

정규표현식 기호들 (1)

표현	설명
*	0개 또는 그 이상 예) $x^* = x$ 가 0번 또는 그 이상 반복
+	1개 또는 그 이상 예) x^+
?	0 또는 1개 (1개 이하) 예) $x^?$
.	무엇이든 한글자
^	시작 문자를 지정, 예) $^{\wedge}[abc] = abc$ 중 한글자 포함해서 시작
[]	해당 문자를 제외한 모든 것, 예) $[^abc] = a,b,c$ 만 빼고 모두
\$	끝 문자 예) $x\$ = x$ 로 종료
[가-힣]	모든 한글 글자 중 1개
	또는
()	소괄호에 묶인 문자는 한개의 그룹으로 표현되거나 and로 됨

정규표현식 기호들 (2)

`\d` - 숫자와 매치, `[0-9]`와 동일한 표현식이다.

`\D` - 숫자가 아닌 것과 매치, `[^0-9]`와 동일한 표현식

`\s` - whitespace 문자와 매치, `[\t\n\r\f\v]`와 동일한 표현식이다.

맨 앞의 빈 칸은 공백문자(space)를 의미

`\S` - whitespace 문자가 아닌 것과 매치, `[^\t\n\r\f\v]`와 동일한 표현식

`\w` - 문자+숫자(alphanumeric)와 매치, `[a-zA-Z0-9_]`와 동일한 표현식

`\W` - 문자+숫자(alphanumeric)가 아닌 문자와 매치, `[^a-zA-Z0-9_]`와 동일한 표현식

`\n` - 줄바꿈 문자

`\t` - 탭 문자

`\v` - 수직 탭 문자

`\b` - 단어 경계 : 백스페이스는 반드시 `[\b]`로 입력해야 한다.

`\c` - 제어 문자

`\f` - 폼 피드 문자

`\x` - 16진수 값

`\0` - 8진수 값

특수표현 : “[: :]” 형태로 존재하는 특수한 표현식이 존재

[:]	의미
[digit:]	숫자
[alpha:]	문자
[lower:]	소문자
[upper:]	대문자
[alnum:]	문자+숫자
[punct:]	기호
[graph:]	문자+숫자+기호
[space:]	띄어쓰기
[blank:]	띄어쓰기+탭

이어지거나 이어지지 않는 문자열

이어지는 문자열 (==)

- 가나다(==a) : 뒤에 “a”가 있는 “가나다”

이어지지 않는 문자열 (!=)

- 가나다(!=b) : 뒤에 “b”로 이어지지 않는 “가나다”

앞에서 이어지는 문자열 (<=)

- (<=c)가나다 : 앞에 “c”가 있는 “가나다”

앞에서 이어지지 않는 문자열 (<!=)

- (<!=c)가나다 : 앞에 “c”가 없는 “가나다”

E-mail

```
^[0-9a-zA-Z]([-_\.]?[0-9a-zA-Z])*@[0-9a-zA-Z]([-_\.]?[0-9a-zA-Z])*\.[a-zA-Z]{2,3}$
```

‘시작을’ 0~9 사이 숫자 or a-z A-Z 아무거나로 시작하고

중간에 - _ . 같은 문자가 있을 수도 있고 없을 수도 있으며

그 후에 0~9 사이 숫자 or a-z A-Z 중 하나의 문자가 없거나 연달아 나올수 있으며

@ 가 반드시 존재하고

0-9a-zA-Z 여기서 하나가 있고

중간에 - _ . 같은 문자가 있을수도 있고 없을수도 있으며,

그 후에 0~9 사이 숫자 or a-z A-Z 중 하나의 문자가 없거나 연달아 나올수 있으며

받드시 . 이 존재하고, [a-zA-Z] 의 문자가 2개나 3개가 존재하면서 종료

탐욕적 및 게으른 수량자

정규 표현식에서는 일치하는 패턴을 찾는 횟수 제한이 없어 필요 이상의 상황을 연출하기도 하는데 이것은 의도적으로 수량자를 탐욕적으로 만들었기 때문이다. 문법에서 말하는 탐욕적 수량자(Greedy Quantifier)란 가능하면 가장 큰 덩어리를 찾는다는 뜻이다. 반대의 개념인 게으른 수량자(Lazy Quantifier)는 패턴에 근접하는 최소한의 덩어리를 찾는다.

- 탐욕적 수량자: *, +, {n,}
- 게으른 수량자: *?, +?, {n,}?

실제 사례 - Pathology report

병리결과지에서 추출

```
\d{8}[\w\W]*?(?=\d{8}|$)
```

8개의 숫자 (병록번호)로 시작하는 텍스트 8개의 병록번호 전까지 또는 마지막 전까지 모든 문자 선택

II.C.2 R에서 사용하는 함수들

grep

정규표현식을 사용해 패턴에 일치하는 것을 찾아오는 함수

```
grep(pattern, x, ignore.case = FALSE, perl = FALSE, value = FALSE, fixed = FALSE, useBytes = FALSE, invert = FALSE)
```

pattern : 정규표현식

x : 데이터

해당 인덱스로 찾아주기도 하고

```
text <- c('a','ab','acb','accb','acccb','accccb')
grep('a',text)
```

```
[1] 1 2 3 4 5 6
```

```
grep('acb',text)
```

```
[1] 3
```

해당 값으로 찾아주기도 함

```
grep('ac?b',text, value=T)
```

```
[1] "ab" "acb"
```

```
grep('ac{2,3}',text,value=T)
```

```
[1] "accb" "acccb" "accccb"
```

gsub

문자열을 바꿔주는 함수

```
gsub(pattern, replacement, x, ignore.case = FALSE, perl = FALSE, fixed = FALSE, useBytes = FALSE)
```

regexr, gregexr(regexr의 global 버전)

grep()과 grepl()의 한계점 보완: 특정 문자 패턴의 일치여부에 대한 정보를 제공하지만 위치 및 정규식의 일치 여부를 알려주지는 않음

```
regexr(pattern, text, ignore.case = FALSE, perl = FALSE, fixed = FALSE, useBytes = FALSE)
```

```
x <- c("Darth Vader: If you only knew the power of the Dark Side.
      Obi-Wan never told you what happend to your father",
      "Luke: He told me enough! It was you who killed him!",
      "Darth Vader: No. I'm your father")
regexr("you", x) # 각 x의 문자열에서 you가 처음 나타난 위치 및 길이 반환
```

```
[1] 17 33 22
attr(,"match.length")
[1] 3 3 3
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
```

```
regexr("father", x) # 패턴을 포함하지 않은 경우 -1 반환
```

```
[1] 111 -1 27
attr(,"match.length")
[1] 6 -1 6
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
```

```
gregexr("you", x) # 각 x의 문자열에서 you가 나타난 모든 위치 및 길이 반환
```

```
[[1]]
[1] 17 86 106
attr(,"match.length")
[1] 3 3 3
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
```

```
[[2]]
[1] 33
attr(,"match.length")
[1] 3
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
```

```
[1] TRUE

[[3]]
[1] 22
attr(,"match.length")
[1] 3
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
```

II.C.3 *stringr package*

str_match

`str_match(string, pattern)` Return the first pattern match found in each string, as a matrix with a column for each () group in pattern. 일치하는 단어를 그룹별로 매트릭스 형식으로 돌려주기 때문에 실제 사용시에 편리함

[stringr cheat sheet](#) 참고

II.D Survival Analysis 심화학습 : 생존예측 모델 만들기

II.D.1 기초개념

예측모델을 만드는 이유?

통계분석만 하면 되는데 왜 예측 모델을 만드나요? : 사실 예측모델을 만드는 기술이나 지식은 통계분석에 필요한 지식이나 기술과 크게 다르지 않습니다.

“림프절 전이가 예후에 미치는 영향”, “성장호르몬이 키의 크기에 미치는 영향”과 같은 분석은 단일 독립변수와 결과와의 인과 관계를 보는 것인데, 예측 모델은 모든 가능한 독립변수들을 다 모아서 분석하는 것이 다르고 복잡할 뿐입니다.

하지만 이러한 독립변수들과 최종결과들간의 상관관계를 연결하는 예측모델을 만들게 되면 전체적인 인과관계를 이해하거나 새로운 연구방법을 고안하는 데에도 도움이 될 것입니다.

Mean, Median 과 standard error와의 관계

표본데이터와의 오차를 SE(standard error)라고 하여 이 값이 작을 수록 예측모델이 정확해짐

선형회귀 모델에서는 표준오차의 제곱이 최소화되는 모델을 최적화 모델로 간주함

일반적으로 1차원적인 데이터에서 표준오차의 절대값의 1승이 최소가 되도록 하는 값은 median이며, 2승이 최소가 되도록 하는 값은 mean, n승(무한대)로 수렴할수록 중간값(최대와 최소의 가운데값)이 된다.

아래는 시뮬레이션을 해본 것

```
set.seed(1234)
data_x <- rnorm(20,10,4)
data_x <- data_x^(1.5)
data_x
```

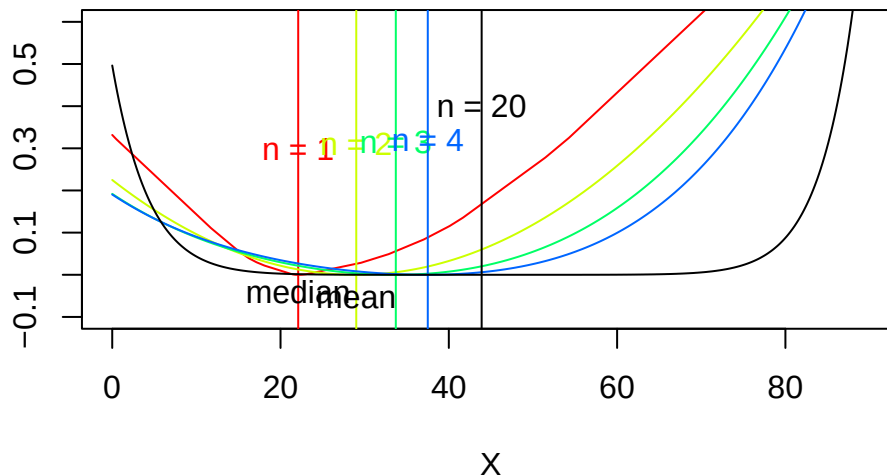
```
[1] 11.7612815754 37.0300665424 54.2902915015 0.4848959707 40.1048388956
[6] 41.6951522361 21.3709624995 21.8406791221 21.5424903066 16.3423118794
[11] 23.0155710831 14.7206591535 18.1050585616 32.8536465579 51.4764834143
[16] 29.5535064769 22.4408368488 16.0212247549 17.1538301508 87.1938705899
```

```
probe_x <- seq(0,90,by=0.1)
sse_m <- NULL
sse_which <- NULL
for (n in 1:100) {
  sse <- NULL
  for (i in probe_x) {
    sse <- c(sse,sum(abs(i-data_x)^n))
    maxsse <- max(sse)
    minsse <- min(sse)
    tempsse <- (sse-minsse)/(maxsse-minsse)
  }
  sse_m <- cbind(sse_m,tempsse)
  sse_which <- c(sse_which, mean(which(sse==min(sse),arr.ind = T)) )
}
plot(probe_x, sse_m[,1], type= "l", xlab = "X",
```

```

      ylab = "", ylim = c(-0.1,0.6), col=rainbow(5)[1])
abline(v=sse_which[1]/10,col=rainbow(5)[1] )
text(sse_which[1]/10,0.3, labels = paste("n =",1), col=rainbow(5)[1])
for (n in 2:4) {
  lines(probe_x, sse_m[,n], type= "l", xlab = "X",
        ylab = "", ylim = c(-0.1,1.1), col=rainbow(5)[n])
  abline(v=sse_which[n]/10,col=rainbow(5)[n] )
  text(sse_which[n]/10,0.3+n/200, labels = paste("n =",n), col=rainbow(5)[n])
}
lines(probe_x, sse_m[,20], type= "l", xlab = "X",
      ylab = "", ylim = c(-0.1,1.1), col="black")
abline(v=sse_which[20]/10,col="black")
text(sse_which[20]/10,0.4, labels = paste("n =",20), col="black")
text(sse_which[1]/10,-0.04, labels = "median", col="black")
text(sse_which[2]/10,-0.06, labels = "mean", col="black")

```



Regression model이란?

종속변수에 영향을 주는 독립변수들을 고려하여 종속변수를 예측할 수 있는 모델을 구하는 것
 평균값과 오차항을 구하여 예측 할 수 있도록 함 - residual (잔차)를 구하여 검증하는 것까지 필요
 검증과정을 거쳐 underfitting, overfitting을 걸러내야 현실세계의 예측가능성을 높일 수 있음
 임상적으로는 진단적 가치와 예후예측의 가치에서 중요한 역할을 함

Randomized Clinical Trial을 할 수 있으면 좋지만 대부분의 경우에는 그렇지 못하며, 이런 경우 bias가 컨트롤 되어야 하는데, 이때 regression model이 수학적으로 컨트롤 해줄 수 있다. 또한 한가지 변수만을 비교하기 위한 RCT라 하더라도 상대적 위험도나 독립변수들의 모델 설명력의 크기를 알기 위해서 regression model이 필요하다.

Linear regression model

독립변수와 종속변수간의 관계가 선형이라고 가정하고 모델을 설정함 (R function : lm)

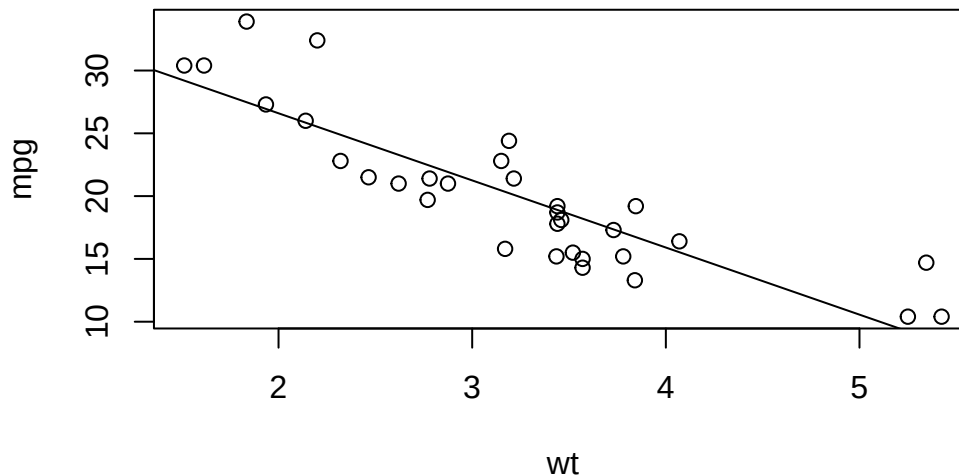
$$\hat{y} = \beta_0 + \sum_{i=1}^p \beta_i x_i$$

```
data("mtcars")
t.test(mpg~am,data = mtcars)
```

Welch Two Sample t-test

```
data: mpg by am
t = -3.7671231, df = 18.332252, p-value = 0.001373638
alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
95 percent confidence interval:
 -11.280194355 -3.209684187
sample estimates:
mean in group 0 mean in group 1
 17.14736842    24.39230769
```

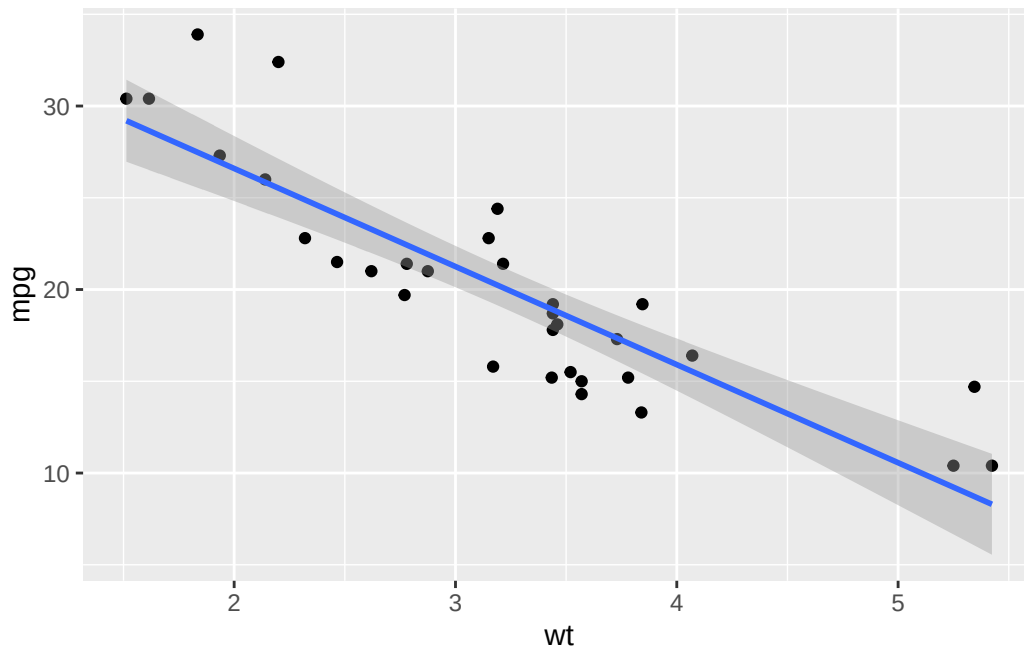
```
fit <- lm(mpg ~ wt, data=mtcars)
plot(mpg ~ wt, data=mtcars)
abline(fit)
```



R에서는 독립변수와 종속변수의 관계를 “종속변수 ~ 독립변수” 형태의 formula로 표현한다.

ggplot에서는 그래프 옵션에 lm 옵션을 넣어서 그래프를 그리면 95% 신뢰구간의 regression 그래프를 그려준다.

```
library(ggplot2)
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  stat_smooth(method = "lm")
```



Non-linear regression model

독립변수와 종속변수간의 관계가 비선형이라고 가정하는데, 다항식으로 가정하고 회귀하는 경우와 일반 smooth 함수를 적용한 일반화 가법 모형이 있음

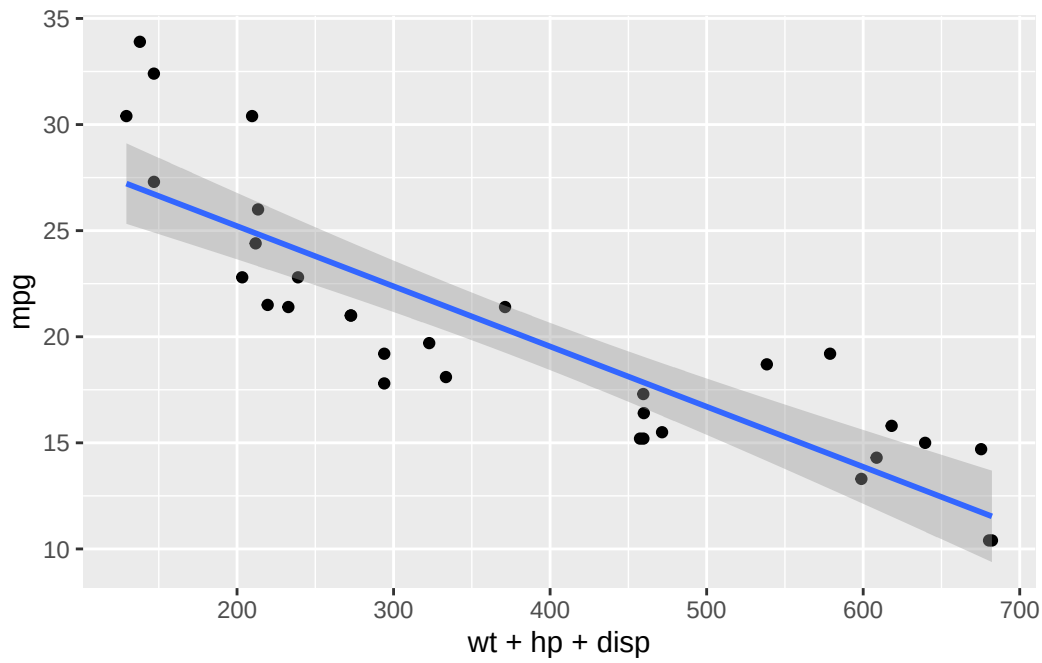
polynomial regression (R function : `lm`)

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots + \beta_k x^k$$

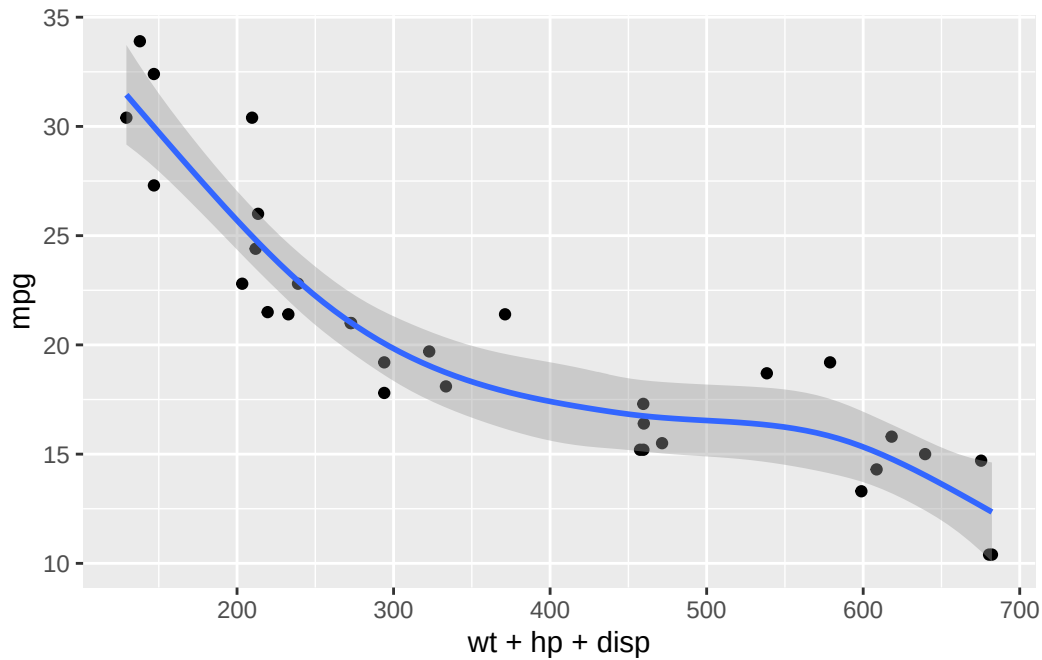
generalized additive model (GAM - R function : `gam`)

$$\hat{y} = \beta_0 + f_1(x_1) + f_2(x_2) + f_3(x_3) + \dots + f_p(x_p) \quad (f : \text{smooth function})$$

```
ggplot(mtcars, aes(x = wt+hp+disp, y = mpg)) +
  geom_point() +
  stat_smooth(method = "lm")
```



```
ggplot(mtcars, aes(x = wt+hp+disp, y = mpg)) +  
  geom_point() +  
  stat_smooth(method = "gam")
```



Generalized Linear Model

오차항의 확률분포가 정규분포가 아닌 경우에 적용 (R function : glm)

$$g(\hat{y}) = \beta_0 + \sum_{i=1}^p \beta_i x_i \quad (g : \text{link function})$$

종속변수에 적절한 함수를 적용하기 때문에 일반화된 모든 선형회귀를 다 적용할 수 있음 - 가장 많이 사용

link function의 종류 : binomial 오차항은 logit, exponential 오차항은 inverse, poisson 오차항은 log

종속변수가 0,1 이분법적인 경우에는 logistic regression, 종속변수가 Ordinal인 경우는 ordinal regression, 종속변수가 count인 경우에는 poisson regression

```
fit_glm <- glm(mpg ~ wt+hp+disp, data=mtcars)
summary(fit_glm)
```

Call:

```
glm(formula = mpg ~ wt + hp + disp, data = mtcars)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	37.1055052690	2.1108152455	17.57876	< 2e-16 ***
wt	-3.8008905826	1.0661906391	-3.56493	0.001331 **
hp	-0.0311565508	0.0114357943	-2.72448	0.010971 *
disp	-0.0009370091	0.0103497449	-0.09053	0.928507

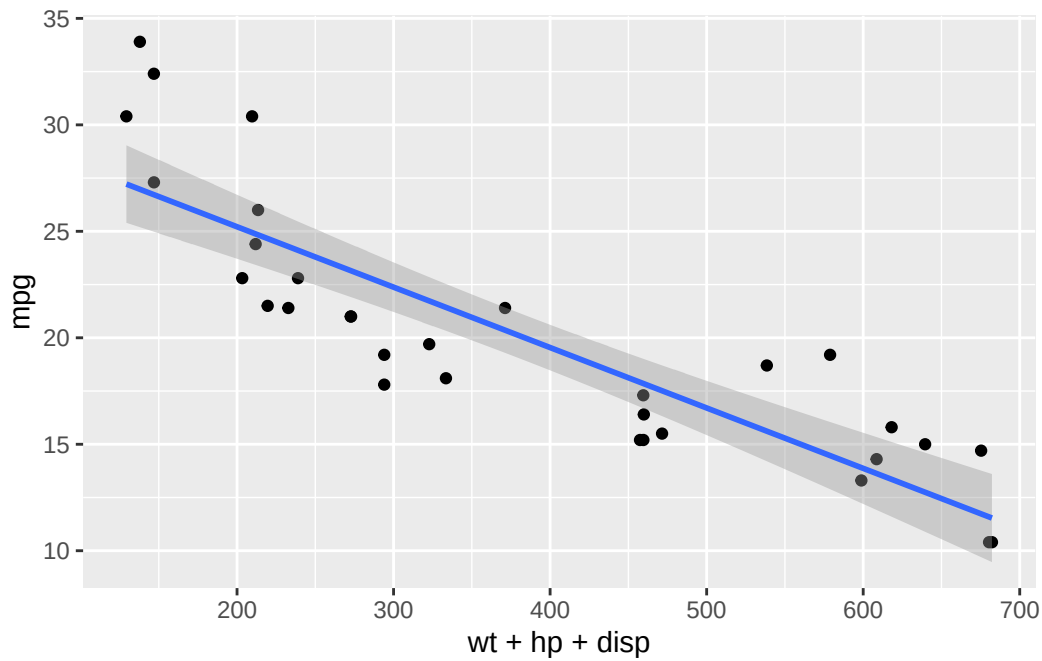
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 6.963952669)

Null deviance: 1126.04719 on 31 degrees of freedom
 Residual deviance: 194.99067 on 28 degrees of freedom
 AIC: 158.64297

Number of Fisher Scoring iterations: 2

```
ggplot(mtcars, aes(x = wt+hp+disp, y = mpg)) +
  geom_point() +
  stat_smooth(method = "glm")
```



```
fit_glmlog <- glm(mpg ~ wt+log(hp)+log(displ), data=mtcars)
summary(fit_glmlog)
```

Call:

```
glm(formula = mpg ~ wt + log(hp) + log(displ), data = mtcars)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	63.8126035	5.6976976	11.19972	7.4539e-12 ***
wt	-2.2783608	0.9213329	-2.47290	0.019739 *
log(hp)	-4.1506167	1.7442588	-2.37959	0.024384 *
log(displ)	-3.0519288	2.1093289	-1.44687	0.159036

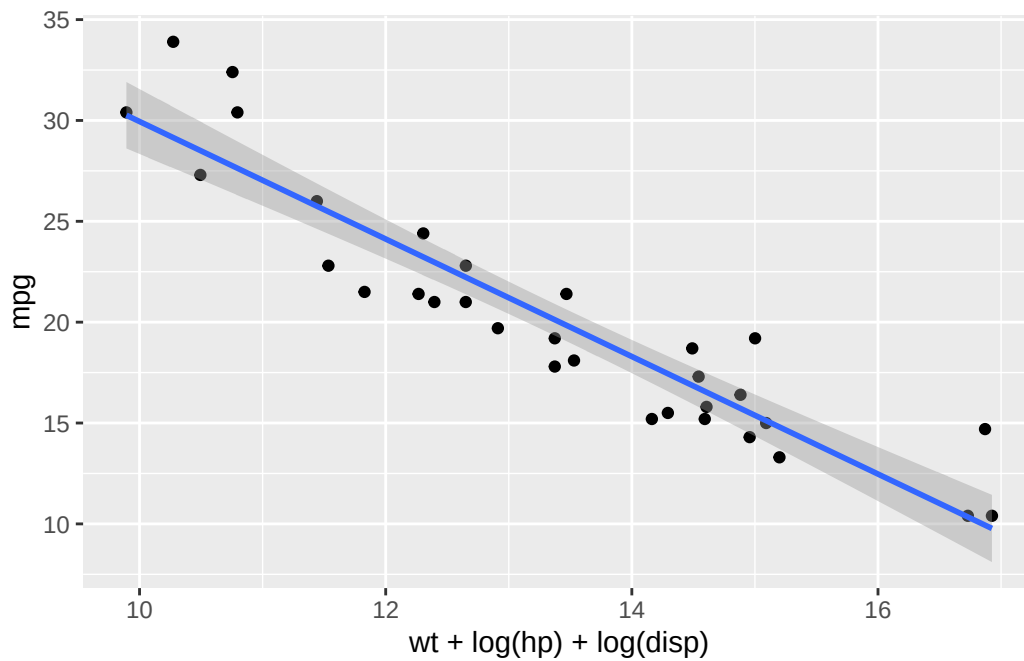
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 5.270595599)

Null deviance: 1126.04719 on 31 degrees of freedom
 Residual deviance: 147.57668 on 28 degrees of freedom
 AIC: 149.72765

Number of Fisher Scoring iterations: 2

```
ggplot(mtcars, aes(x = wt+log(hp)+log(displ), y = mpg)) +
  geom_point() +
  stat_smooth(method = "glm")
```



Nominal Predictors

명목변수가 독립변수로 들어가는 경우가 있다. 예를 들면 네가지 치료법, J, K, L, M 등이 있고 이로 인하여 T라는 효과가 발생한다고 하면 이 모델은 다음과 같이 표현된다.

$$C(Y|T = J) = \beta_0$$

$$C(Y|T = K) = \beta_0 + \beta_1$$

$$C(Y|T = L) = \beta_0 + \beta_2$$

$$C(Y|T = M) = \beta_0 + \beta_3$$

Interactions

두개의 독립변수가 완전히 독립적이지 않고 상호의존적인 경우가 있다. 독립변수 X_1, X_2 가 서로 상호작용을 하는지 알아보려면 $X_1 \times X_2$ 항의 회계계수 t 검정을 통해서 판정하면 된다.

$X_1 X_2$ 항과 같은 항이 한개면 어렵지 않으나 여러개가 있으면 매우 복잡해짐. - 이런 경우에는 독립변수에 대한 임상적인 경험으로 판단하는 수밖에 없음.

연속변수는 범주화하지 않도록 주의

범주화를 하게 되는 이유들

연속변수에 의한 효과가 선형이 아니고 불규칙하기 때문에 구간으로 나누고 싶어하는 경향이 발생함. 특히 불연속 변수에 의한 상호작용이 있을 경우에 가능성이 높아진다. 범주화의 딜레마 - 범주화한 다음에 모델의 적합도를 높이려면 구간을 세밀하게 나눠야 하는 문제가 발생하는데 그러면 교란변수의 문제나 샘플 크기의 문제가 발생함.

간단하게 비선형화 항을 넣는 방법

불규칙한 상관관계를 표현하기 위한 여러가지 방법중 하나는 제곱항을 넣는 방법이다.

$$C(Y|X) = \beta_0 + \beta_1 X_1 + \beta_2 X_1^2$$

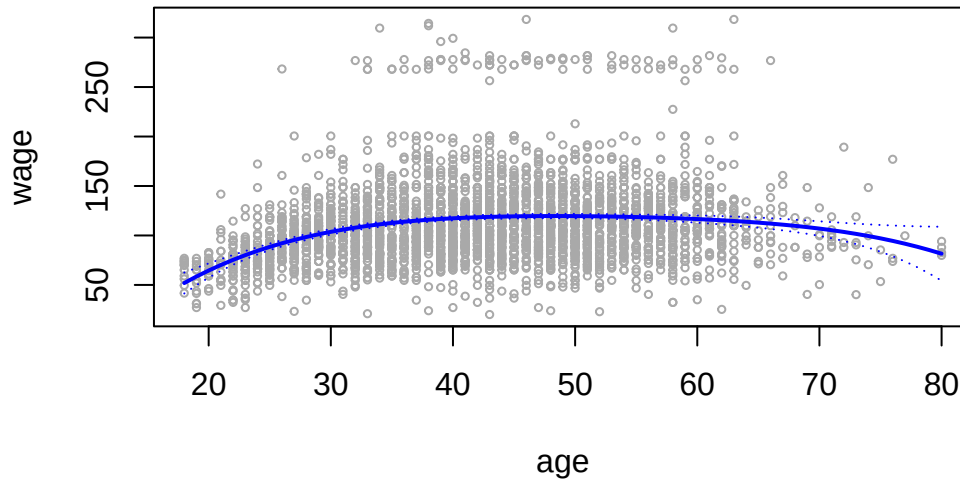
이 식이 진짜로 선형이라면 β_2 의 값은 0이 될 것이다. 때로는 제곱항이 아니라 다항식으로 넣기도 하고 $\log(X)$ 변형을 넣기도 한다. 예측모델을 만들 때에는 진짜 모델을 모르기 때문에 다양한 시도를 해보고 검증하고 경험적으로 적절한지 따져봐야 함

```
library(ISLR)
attach(Wage)
fit=lm(wage~poly(age,4),data=Wage)
coef(summary(fit))
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	111.70360820	0.7287409385	153.283014990	0.000000000e+00
poly(age, 4)1	447.06785276	39.9147850600	11.200557690	1.484604276e-28
poly(age, 4)2	-478.31580585	39.9147850600	-11.983424316	2.355830850e-32
poly(age, 4)3	125.52168603	39.9147850600	3.144741625	1.678621781e-03
poly(age, 4)4	-77.91118099	39.9147850600	-1.951937882	5.103864983e-02

```
agelims=range(age)
age.grid=seq(from=agelims[1],to=agelims[2])
preds=predict(fit,newdata=list(age=age.grid),se=TRUE)
se.bands=cbind(preds$fit+ 2*preds$se.fit, preds$fit-2*preds$se.fit)
plot(age,wage,xlim=agelims,cex=.5,col="darkgrey")
title("Degree-4 Polynomial",outer=T)
lines(age.grid,preds$fit,lwd=2,col="blue") #plot 위에 라인 그리는거
matlines(age.grid,se.bands,lwd=1,col="blue",lty=3)
```

Degree-4 Polynomial



```
detach(Wage)
```

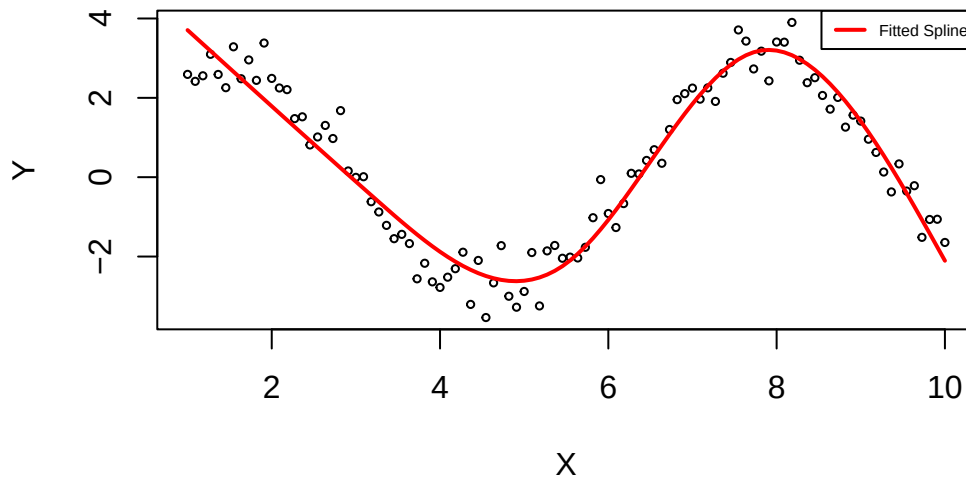
Splines 를 넣어서 불규칙한 회귀 예측모델을 만드는 방법

직선의 방정식이 일정한 점에서 꺾이도록 만들 수 있다. 꺾이는 점들을 knot라고 한다.

불규칙하게 꺾이는 형태의 데이터도 선형회귀 예측모델을 만들 수 있다. 하지만 이것은 그야말로 직선만 되기 때문에 좀 더 완만한 곡선의 형태로 만들기 위하여 3차함수를 이용한 Cubic splines를 사용한다. k개의 knot를 설정할 경우 k+3개의 regression coefficient 를 갖게 된다.

```
library(splines)
x <- seq(1, 10, length.out = 100)
y <- 3 * sin(x) + rnorm(100, mean = 0, sd = 0.5)
spline_model <- lm(y ~ ns(x, df = 4)) ## ns = natural cubic splines, df = knot숫자
plot(x, y, main = "Spline Regression Example", xlab = "X", ylab = "Y", cex=0.5)
lines(x, predict(spline_model), col = "red", lwd = 2)
legend("topright", legend = "Fitted Spline", col = "red", lwd = 2, cex=0.5)
```

Spline Regression Example



불규칙한 Splines를 해결하는 Restricted Cubic Splines

Stone and Koo는 natural cubic splines가 양쪽 끝에서 문제가 있다는 것을 알고 $k+3$ 대신 $k-1$ parameter만 사용함. 이렇게 하면 knot의 범위를 벗어난 경우에도 어느 정도 extrapolation이 허용된다는 장점이 있다. 또한 fixed quantile 방법으로 knot의 위치를 정해 놓더라도 그 위치에 따른 모델의 변화가 민감하지 않다고 함. 따라서 R package의 rcs 함수 사용시 default 값을 사용해도 무방하다.

Nonparametric Regression : LOESS

비모수회귀모델도 완전한 곡선으로 만드는 것이 가능하다. locally weighted least squares method 라는 방법을 사용하여 일반적인 연속 변수와 범주형 변수의 선형적인 경향성을 보여주는 예측모델을 만드는 것이 가능함. Gray 등은 10개의 고정 knot를 이용해 비모수적인 spline과 회귀적 spline 중간적인 방법을 사용하는 일반적인 방법을 고안해 내었다. 이를 효율적으로 사용할 수 있다.

Parametric Regression Splines를 써야 하는 이유

기존의 모델에 수식의 형태로 끼워넣는 것이 가능하다.

R의 표준적인 formula에 집어 넣어서 분석할 수 있다.

쉽게 연산이 되므로 예측함수로 만드는 것도 용이하다.

Recursive Partitioning: Tree-based Models

CART (classification and regression tree) 라고 부르는 일종의 재귀적 tree 구조 분류식 예측모델

regression 모델하고는 상관이 없지만 분류를 할 때 분류하는 기준을 regression 하는 기준으로 분류한다고 해서 regression tree라는 이름을 붙임

서로 다른 그룹의 최적의 분리점을 찾아서 분리하는 과정을 반복해내면 동일한 집단을 예측할 수 있는 변수들을 알 수 있게 됨.

검증하기

Bootstrap

Bradley Efron's bootstrap은 데이터의 분포에 대한 아무런 가정이 필요없는 평가 시스템. Training할 데이터set 에서 복원추출을 하여 모델링을 하는 과정을 반복하게 되었을 경우, 그것이 실제 데이터로 모델링을 하는 것에 근접하게 될 것이라는 추정이 가능함

sampling 횟수 B는 최소한 200번 이상이어야 하며 신뢰구간을 크게 하려면 더욱 커져야 함. (~1000)

Data splitting

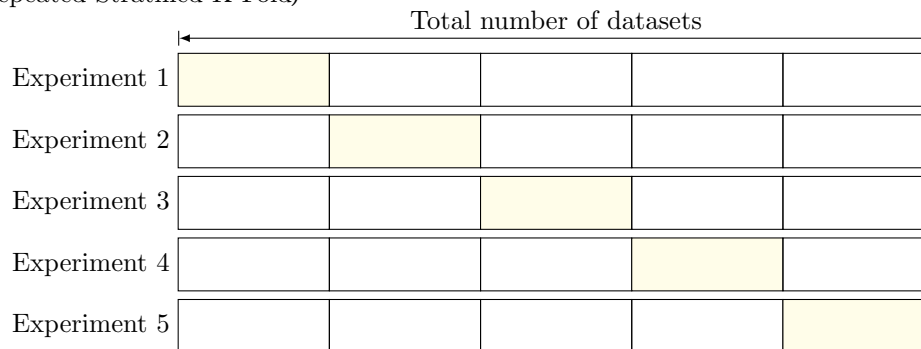
External validation이 더 객관적이어서 좋을 것 같지만 외부 검증할 때에 사용되는 방법들이 상호호환이 안될 가능성등 여러가지 문제점들이 있어서 실제로는 내부 검증법이 수학적으로는 더 안정적인 가능성이 높음. internal validation 방법의 하나로 데이터를 두개로 나누어서 하나는 training set, 다른 하나는 validation set로 사용하는 것이 자주 사용됨

데이터를 쪼개는 것은 단점이 있다. 더 많은 데이터를 요구하게 되고, 막상 전체 데이터로는 모델을 만들 수 없다. (training set로 만든 모델만 존재) 또한 데이터 쪼개기가 잘되면 우연히 너무 좋은 예측모델이 만들어질 수도 있다.

Resampling : Data-splitting의 개선

데이터 쪼개기의 개선을 위해서 결국 resampling 방법을 사용한다.

cross-validation은 k=10 이라면 10개의 그룹으로 나누고 9개의 그룹으로 모델을 만든 다음 나머지 한개의 그룹으로 검증 이러한 과정을 돌아가면서 10회 반복한다. 반복횟수를 늘리고 싶으면 k=10 으로 나누는 것을 20~50회 무작위로 할 수도 있다. (Repeated Stratified K-Fold)



cross validation은 data splitting의 일반적인 해법이지만 항상 일정 부분은 모델 만들기에서 빠져야 한다. 원래 모든 데이터를 다 사용했을 때 (Full model) 가장 정확하게 예측할 수 있는 모델이 나오는 법이다. 대신 미래에 데이터를 더 많이 모으게 되면 분산이 더 늘어날 위험성도 있다.

결론 : Bootstrap을 이용한 validation

원래의 데이터 X에서 Bootstrap방법으로 sample n개(원래의 데이터 갯수와 같은 갯수)를 복원추출해서 모델을 만들고 원래의 데이터에 적용한 것과의 차이를 accuracy index로 삼는다. 이 과정을 100번이상 반복하여 평균하고 final model fit의 정확도에서 빼서 overfitting을 교정한다.

The bootstrap is a great way to estimate the performance of a model. The simplest thing to estimate is variance. More to your original point, the bootstrap can estimate the likely future performance of a given modeling procedure, on new data not yet realized. - Frank Harrell answered

원래의 데이터에서 복원추출로 원래의 데이터와 같은 갯수를 샘플하면 완전히 동일한 데이터가 나오는 것이 아니고 평균적으로 $1 - e^{-1} = 0.632$ 만큼의 데이터만 선택됨. 일부는 2, 3, 4.. 차례 선택될 수 있다. 반복된 (B 횟수) Bootstrapping을 하게 되면 원래의 데이터의 모집단의 데이터 분포와 유사한 미래의 데이터 셋트를 대상으로 테스트하는 효과가 나온다고 함. (수학적 검증은 넘어감)

R model language : formula and fitting

기본적인 formula는 response ~ terms의 형태

```
1 class ~ cp + age + sex
2 class ~ cp + age + sex + age*sex # add interaction
```

```

3 class ~ (cp + age + sex)^2 # all effects
4 f <- lrm (y ~ rcs (x ,4) + x2 + x3 )
5 f2 <- update (f , subset =sex == " male ")
6 f3 <- update (f , ~..-x2 ) # remove x2 from model
7 f4 <- update (f , ~.. + rcs (x5 ,5)) # add rcs (x5 ,5) to model
8 f5 <- update (f , y2 ~ .) # same terms , new response var.

```

model에서 독립변수들의 상호작용, spline, polynomial 함수나 log 함수 등의 적용, subset 제한, update 등의 변형등도 가능함

Table 8: rms Fitting Functions

Functions	Purpose	related Functions
ols	Ordinary least squares linear model	lm
lrm	Binary and ordinal logistic regression model, Has options for penalized MLE	glm
psm	Accelerated failure time parametric survival models	survreg
cph	Cox proportional hazards regression	coxph
bj	Buckley–James censored least squares model	survreg, lm
Glm	General linear models	glm
Gls	Generalized least squares	gls
Rq	Quantile regression	rq

Table 9: rms Transformation Function

Functions	Purpose	related Function
asis	원래 그대로 - 쓸 필요가 없음	I
rcs	Restricted cubic spline	ns
pol	Polynomial using standard notation	poly
lsp	Linear spline	
catg	Categorical predictor (seldom) factor는 자동설정됨	factor
scored	Ordinal categorical variables 명목변수도 자동설정	ordered
matrx	anova and fastbw 를 위해 변수를 행렬로 보관함 (자동)	matrix
strat	Nonmodeled stratification factors (used for cph only)	strata

Table 10: rms Package Functions

Functions	Purpose
specs	Detailed specifications of fit
lrtest	Likelihood ratio test for two nested models ex) lrtest(fit1, fit2)
plot.summary.rms	Plot continuously shaded confidence bars for results of summary
plot.anova.rms	Graphical depiction of anova
Predict	Predicted values and confidence limits easily varying a subset of predictors and leaving the rest set at default values
plot.Predict	Plot the result of Predict using lattice
ggplot.Predict	Plot the result of Predict using ggplot2
bplot	3-dimensional plot when Predict varied two continuous predictors over a fine grid
gendata	Easily generate predictor combinations
predict	Obtain predicted values or design matrix
fastbw	Fast backward step-down variable selection
residuals, resid	Residuals, influence stats from fit
vif	Computes variance inflation factors from the covariance matrix 분산팽창요인 - 다중회귀분석에서 독립변수간의 상관관계를 측정하는 척도
sensuc	Sensitivity analysis for unmeasured confounder - 이로 인한 편향을 평가
which.influence	Which observations are overly influential
latex	LATEX representation of fitted model
nomogram	Draws a nomogram for the fitted model
validate	Validate indexes of model fit using resampling
survest	Estimate survival probabilities (psm, cph)
survplot	Plot survival curves (psm, cph)

Table 11: rms Misc. functions

Functions	Purpose
Hazard	R function analytic representation of a fitted hazard function (for psm)
Function.transcan	R function analytic representation of $X \hat{\beta}$ from a fitted regression model transcan : nonlinear additive transformation and imputation function
Survival	R function analytic representation of fitted survival function (for psm, cph)

** cut2 function은 연속변수를 구간별 명목변수로 변환하는데 사용될 수 있다.

```
1 Hmisc::cut2(runif(1000,0,100), c(10,30,60)) ## factor (interval)
```

Missing value에 대한 처리규칙

다음과 같이 정할 수 있다. rms에서는 na.delete가 기본옵션

```
1 options ( na.action ="na.omit") # don't report frequency of NAs
2 ## other options
3 ## "na.fail" : Stop with error message if any missing values present
4 ## "na.omit" : Function to remove observations with any missing values
5 ## "na.delete" : Modified version of na.omit – report frequency of each NAs
```

결측치가 생기면 이로 인하여 fit 모델과 실제 데이터간의 행이 일치하지 않는 문제가 생긴다. 이 문제를 해결하는 방법들이 있지만 불편하므로 특별한 경우가 아니면 모델을 만들 최종 데이터셋에서는 결측값이 있는 데이터는 제거하는 것을 원칙으로 하다.

그러나.. 아까운 경우가 있다. 별로 중요하지 않을 것 같은 변수가 결측값인 경우에는 permutation 방법이 있다.

Case Study : Titanic survival prediction

```
getHdata(titanic3)
str(titanic3)
```

```
'data.frame': 1309 obs. of 14 variables:
 $ pclass : Factor w/ 3 levels "1st","2nd","3rd": 1 1 1 1 1 1 1 1 1 ...
 $ survived : 'labelled' int 1 1 0 0 0 1 1 0 1 0 ...
 ..- attr(*, "label")= chr "Survived"
 $ name : 'labelled' chr "Allen, Miss. Elisabeth Walton" "Allison, Master. Hudson Trevor" "Allison, Master. Hudson Trevor" ...
 ..- attr(*, "label")= chr "Name"
 $ sex : Factor w/ 2 levels "female","male": 1 2 1 2 1 2 1 2 1 2 ...
 $ age : 'labelled' num 29 0.917 2 30 25 ...
 ..- attr(*, "units")= chr "Year"
 ..- attr(*, "label")= chr "Age"
 $ sibsp : 'labelled' int 0 1 1 1 1 0 1 0 2 0 ...
 ..- attr(*, "label")= chr "Number of Siblings/Spouses Aboard"
 $ parch : 'labelled' int 0 2 2 2 2 0 0 0 0 0 ...
 ..- attr(*, "label")= chr "Number of Parents/Children Aboard"
 $ ticket : 'labelled' chr "24160" "113781" "113781" "113781" ...
 ..- attr(*, "label")= chr "Ticket Number"
 $ fare : 'labelled' num 211 152 152 152 152 ...
 ..- attr(*, "units")= chr "British Pound (\\243)"
 ..- attr(*, "label")= chr "Passenger Fare"
 $ cabin : Factor w/ 187 levels "", "A10", "A11",...: 45 81 81 81 81 151 147 17 63 1 ...
 $ embarked : Factor w/ 3 levels "Cherbourg", "Queenstown",...: 3 3 3 3 3 3 3 3 1 ...
 $ boat : Factor w/ 28 levels "", "1", "10", "11",...: 13 4 1 1 1 14 3 1 28 1 ...
 $ body : 'labelled' int NA NA NA 135 NA NA NA NA 22 ...
 ..- attr(*, "label")= chr "Body Identification Number"
```

```
$ home.dest: 'labelled' chr "St Louis, MO" "Montreal, PQ / Chesterville, ON" "Montreal, PQ / Chester
..- attr(*, "label")= chr "Home/Destination"
```

```
dd <- datadist(titanic3)
options(datadist = "dd")
attach(titanic3)

#s <- summary(survived ~ age + sex + pclass + cut2(sibsp,0:3) +
#             cut2(parch, 0:3), data = titanic3)
#plot(s, main="Univariate Summaries", subtitles = F, cex=0.8)

f1 <- lrm ( survived ~ sex * pclass * rcs (age ,5) +
           rcs (age ,5) *( sibsp + parch ), data =titanic3 )

anova(f1)
```

	Wald Statistics		Response: survived
Factor		Chi-Square	d.f. P
sex (Factor+Higher Order Factors)	187.14	15	<.0001
All Interactions	59.74	14	<.0001
pclass (Factor+Higher Order Factors)	100.10	20	<.0001
All Interactions	46.50	18	0.0003
age (Factor+Higher Order Factors)	56.20	32	0.0052
All Interactions	34.57	28	0.1826
Nonlinear (Factor+Higher Order Factors)	28.66	24	0.2331
sibsp (Factor+Higher Order Factors)	19.67	5	0.0014
All Interactions	12.13	4	0.0164
parch (Factor+Higher Order Factors)	3.51	5	0.6216
All Interactions	3.51	4	0.4761
sex * pclass (Factor+Higher Order Factors)	42.43	10	<.0001
sex * age (Factor+Higher Order Factors)	15.89	12	0.1962
Nonlinear (Factor+Higher Order Factors)	14.47	9	0.1066
Nonlinear Interaction : f(A,B) vs. AB	4.17	3	0.2441
pclass * age (Factor+Higher Order Factors)	13.47	16	0.6385
Nonlinear (Factor+Higher Order Factors)	12.92	12	0.3749
Nonlinear Interaction : f(A,B) vs. AB	6.88	6	0.3324
age * sibsp (Factor+Higher Order Factors)	12.13	4	0.0164
Nonlinear	1.76	3	0.6235
Nonlinear Interaction : f(A,B) vs. AB	1.76	3	0.6235
age * parch (Factor+Higher Order Factors)	3.51	4	0.4761
Nonlinear	1.80	3	0.6147
Nonlinear Interaction : f(A,B) vs. AB	1.80	3	0.6147
sex * pclass * age (Factor+Higher Order Factors)	8.34	8	0.4006
Nonlinear	7.74	6	0.2581
TOTAL NONLINEAR	28.66	24	0.2331
TOTAL INTERACTION	75.61	30	<.0001
TOTAL NONLINEAR + INTERACTION	79.49	33	<.0001
TOTAL	241.93	39	<.0001

```
detach(titanic3)
```

```
ltx <- function (fit) latex(fit, digits = 3, inline = TRUE,
                             before = "$$X\\hat\\beta=", after = "$$.")
ltx(f1) %>% cat
```

$$\begin{aligned}
& -2.96 \\
& +14.4[\text{male}] \\
& +13.8[2\text{nd}] + 5[3\text{rd}] \\
& +0.516\text{age} - 0.000784(\text{age} - 5)_+^3 + 0.00398(\text{age} - 21)_+^3 - 0.00452(\text{age} - 28)_+^3 \\
& +0.00143(\text{age} - 37)_+^3 - 0.000104(\text{age} - 57)_+^3 - 0.871 \text{ sibsp} - 0.324 \text{ parch} \\
& +[\text{male}][-17.2 [2\text{nd}] - 14.1 [3\text{rd}]] \\
& +[\text{male}][-1.27\text{age} + 0.00168(\text{age} - 5)_+^3 - 0.00791(\text{age} - 21)_+^3 + 0.00842(\text{age} - 28)_+^3 \\
& - 0.00234(\text{age} - 37)_+^3 + 0.000149(\text{age} - 57)_+^3] \\
& +[2\text{nd}][-1.02\text{age} + 0.00123(\text{age} - 5)_+^3 - 0.00474(\text{age} - 21)_+^3 + 0.00368(\text{age} - 28)_+^3 \\
X\hat{\beta} = & +1.65 \times 10^{-8}(\text{age} - 37)_+^3 - 0.00017(\text{age} - 57)_+^3] \\
& +[3\text{rd}][-0.616\text{age} + 0.000831(\text{age} - 5)_+^3 - 0.00426(\text{age} - 21)_+^3 + 0.00513(\text{age} - 28)_+^3 \\
& - 0.00194(\text{age} - 37)_+^3 + 0.000234(\text{age} - 57)_+^3] \\
& +\text{sibsp}[0.00787\text{age} + 4.24 \times 10^{-5}(\text{age} - 5)_+^3 - 0.000111(\text{age} - 21)_+^3 \\
& - 7.34 \times 10^{-5}(\text{age} - 28)_+^3 + 0.000196(\text{age} - 37)_+^3 - 5.41 \times 10^{-5}(\text{age} - 57)_+^3] \\
& +\text{parch}[0.05\text{age} - 7.63 \times 10^{-5}(\text{age} - 5)_+^3 + 0.000219(\text{age} - 21)_+^3 - 5.14 \times 10^{-5}(\text{age} - 28)_+^3 \\
& - 0.000121(\text{age} - 37)_+^3 + 2.99 \times 10^{-5}(\text{age} - 57)_+^3] \\
& +1.11 [\text{male}] \times [2\text{nd}] \times \text{age} + 1.1 [\text{male}] \times [3\text{rd}] \times \text{age} - 3.47 [\text{male}] \times [2\text{nd}] \times \text{age}' \\
& - 3.63 [\text{male}] \times [3\text{rd}] \times \text{age}' + 11.9 [\text{male}] \times [2\text{nd}] \times \text{age}'' + 16.2 [\text{male}] \times [3\text{rd}] \times \text{age}'' \\
& - 6.99 [\text{male}] \times [2\text{nd}] \times \text{age}''' - 17.4 [\text{male}] \times [3\text{rd}] \times \text{age}'''
\end{aligned}$$

Case study: parametric survival model 빌드업과정

```
getHdata ( support ) # Get data frame from web site
acute <- support$dzclass %in% c( 'ARF/MOSF' , 'Coma' )
# support[acute,] %>% naclus %>% plot
ac <- support[acute,]
ac$dzgroup %>% table ## unused levels = 0
```

ARF/MOSF w/Sepsis	COPD	CHF	Cirrhosis
391	0	0	0
Coma	Colon Cancer	Lung Cancer	MOSF w/Malig
60	0	0	86

```
ac$dzgroup <- ac$dzgroup[drop=T] # remove unused levels
attach(ac)
dd <- datadist (ac)
```

datadist (ac)

	age	death	sex	hospdead	slos	d.time
Low:effect	47.93197630	0	<NA>	0	9.0000000	16.000000
Adjust to	63.67398070	0	male	0	15.0000000	182.000000
High:effect	74.49395750	1	<NA>	1	27.0000000	724.000000
Low:prediction	22.19261663	0	female	0	3.0000000	3.000000
High:prediction	89.05311746	1	male	1	110.0744879	1919.223464
Low	18.04199220	0	female	0	3.0000000	3.000000
High	95.50994870	1	male	1	241.0000000	2022.000000

	dzgroup	dzclass	num.co	edu	income	scoma
Low:effect	<NA>	<NA>	1	10	<NA>	0
Adjust to	ARF/MOSF w/Sepsis	ARF/MOSF	1	12	under \$11k	0
High:effect	<NA>	<NA>	2	14	<NA>	37
Low:prediction	ARF/MOSF w/Sepsis	ARF/MOSF	0	4	under \$11k	0
High:prediction	MOSF w/Malig	Cancer	5	18	>\$50k	100
Low	ARF/MOSF w/Sepsis	ARF/MOSF	0	0	under \$11k	0
High	MOSF w/Malig	Cancer	6	22	>\$50k	100

	charges	totcst	totmcst	avtisst	race
Low:effect	27389.000000	15412.421850	14415.39455	19.62499237	<NA>
Adjust to	51079.000000	29308.062500	26322.87500	28.00000000	white
High:effect	100904.000000	57028.062500	54101.67190	39.00000000	<NA>
Low:prediction	8182.357542	4240.832947	2741.71517	10.00000000	white
High:prediction	407524.877095	224558.552374	134399.32751	55.03724395	hispanic
Low	3448.000000	0.000000	0.000000	4.00000000	white
High	740010.000000	390460.500000	271467.25000	64.00000000	hispanic

	meanbp	wbhc	hrt	resp	temp
Low:effect	59	7.9748535150	75.00000000	12	36.39843750
Adjust to	73	12.3984375000	111.00000000	24	37.79687500
High:effect	111	18.1992188000	126.00000000	32	38.50000000
Low:prediction	33	0.0999908447	35.88826816	6	35.09375000
High:prediction	147	41.0648568436	180.00000000	45	40.00174581
Low	0	0.0499954224	0.00000000	0	32.50000000
High	180	100.0000000000	300.00000000	64	41.19531250

	pafi	alb	bili	crea	sod
Low:effect	137.87500000	2.224731445	0.5999755860	0.899902344	134.000000
Adjust to	202.56250000	2.599609380	0.8999023440	1.399902340	137.000000
High:effect	290.00000000	3.099609380	2.0000000000	2.599609380	142.000000
Low:prediction	66.46927374	1.500000000	0.1999816890	0.500000000	126.981378
High:prediction	516.49162011	4.156926504	19.5466858291	9.001854923	154.018622
Low	45.00000000	1.099853520	0.0999908447	0.299987793	118.000000
High	869.37500000	4.799804690	39.2968750000	11.798828100	175.000000

	ph	glucose	bun	urine	adlp
Low:effect	7.379882810	106.00000000	16.750000	1156.500000	0.000000000
Adjust to	7.419921880	141.00000000	30.000000	1870.000000	1.000000000
High:effect	7.469726560	200.00000000	56.000000	2795.000000	3.000000000
Low:prediction	7.189453120	70.53631285	5.000000	0.000000	0.000000000
High:prediction	7.556715185	431.97579143	115.575419	6295.977654	6.081936685
Low	6.959960940	30.00000000	1.000000	0.000000	0.000000000
High	7.659179690	598.00000000	146.000000	7750.000000	7.000000000

	adls	sfdm2	adlsc
Low:effect	0	<NA>	0.00000000
Adjust to	1	<2 mo. follow-up	1.83899975
High:effect	4	<NA>	3.37500000
Low:prediction	0 no(M2 and SIP pres)		0.00000000
High:prediction	7 <2 mo. follow-up		7.00000000
Low	0 no(M2 and SIP pres)		0.00000000
High	7 <2 mo. follow-up		7.00000000

Values:

```

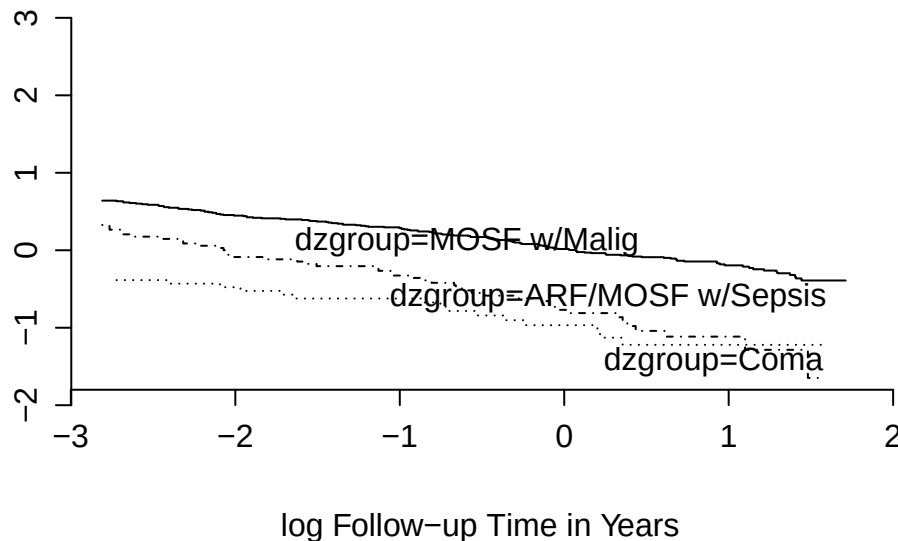
death : 0 1
sex : female male
hospdead : 0 1
dzgroup : ARF/MOSF w/Sepsis Coma MOSF w/Malig
dzclass : ARF/MOSF COPD/CHF/Cirrhosis Coma Cancer
num.co : 0 1 2 3 4 5 6
income : under $11k $11-$25k $25-$50k >$50k
race : white black asian other hispanic
adlp : 0 1 2 3 4 5 6 7
adls : 0 1 2 3 4 5 6 7
sfdm2 :
[1] no(M2 and SIP pres) adl>=4 (>=5 if sur) SIP>=30
[4] Coma or Intub <2 mo. follow-up

```

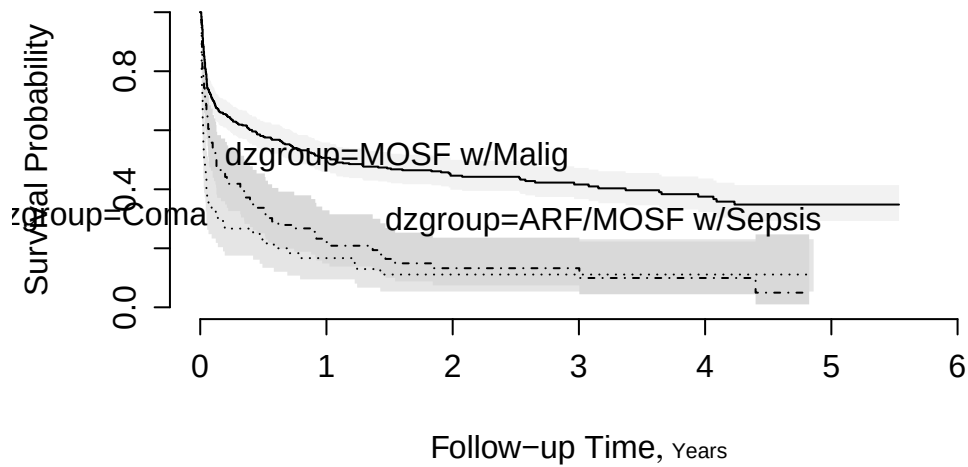
```

options(datadist = 'dd' )
years <- d.time/365.2422
units(years) <- "Year"
S <- Surv(years, death)
survplot ( npsurv (S ~ dzgroup ), conf = "none" ,
           fun = qnorm , logt = T )

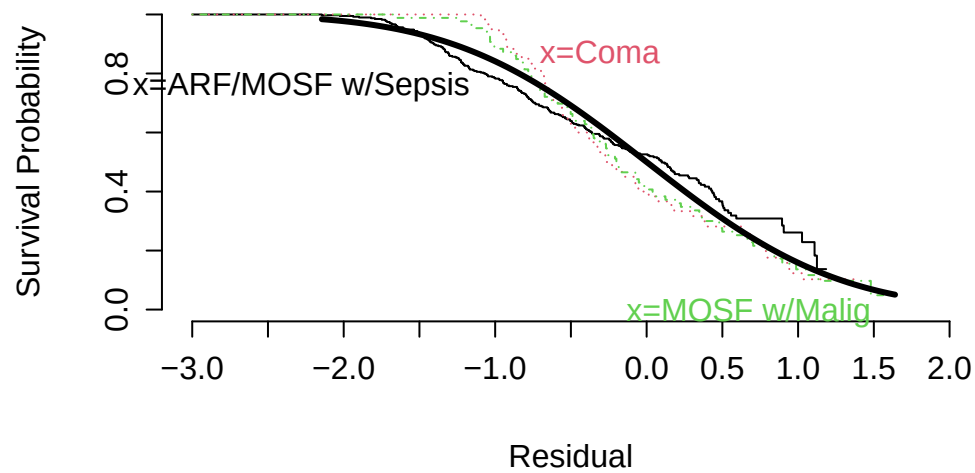
```



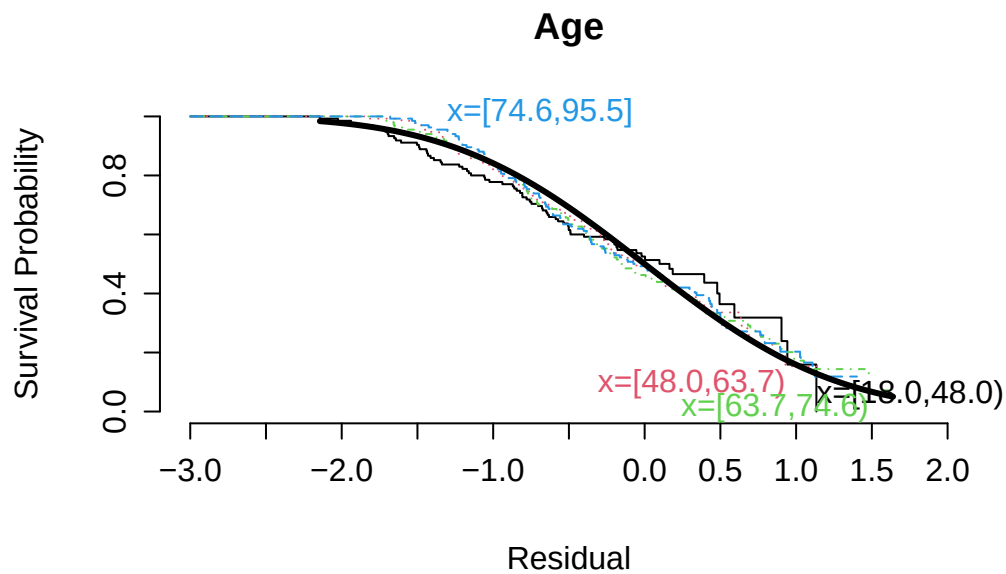
```
survplot(npsurv (S ~ dzgroup ))
```



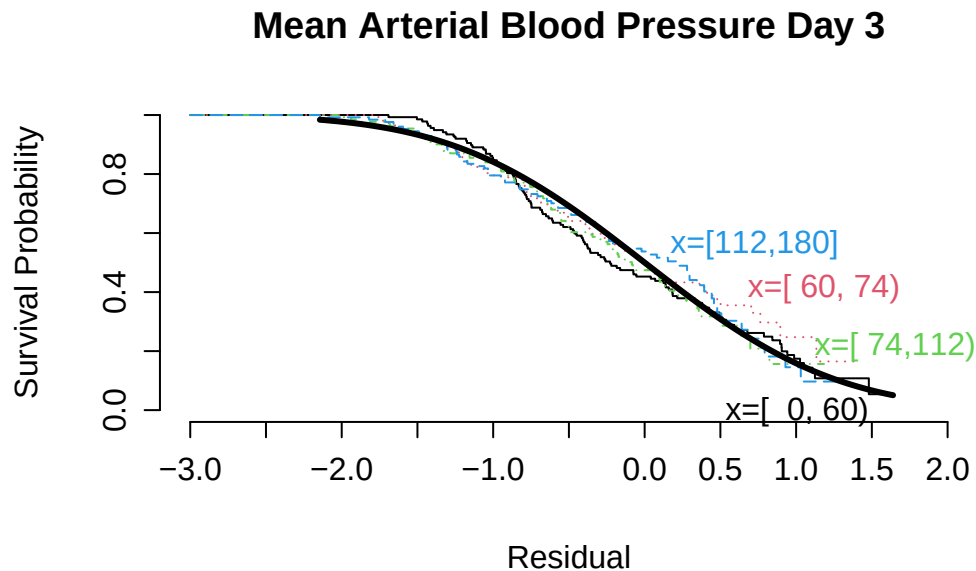
```
f <- psm (S ~ dzgroup + rcs (age ,5) + rcs ( meanbp ,5),
          dist= "lognormal" , y=TRUE )
r <- resid (f)
survplot (r, dzgroup )
```



```
survplot (r, age )
```

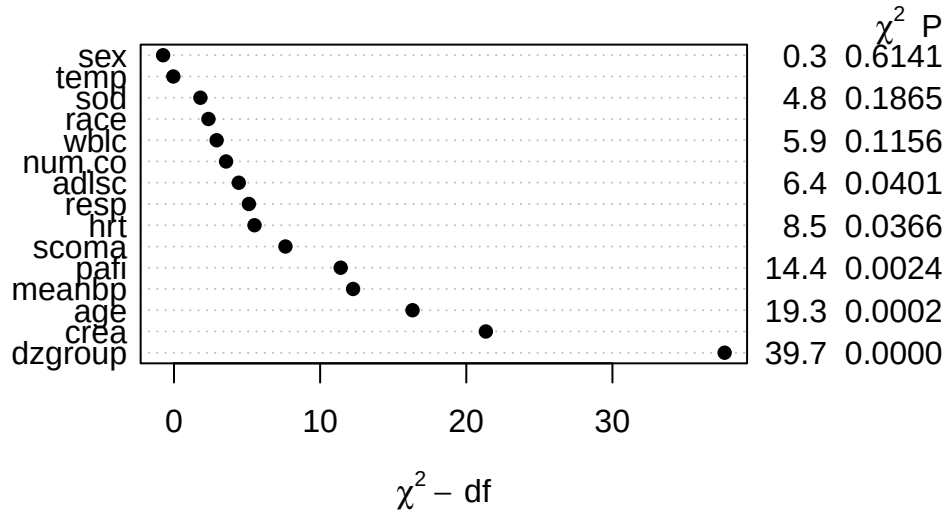


```
survplot (r, meanbp )
```

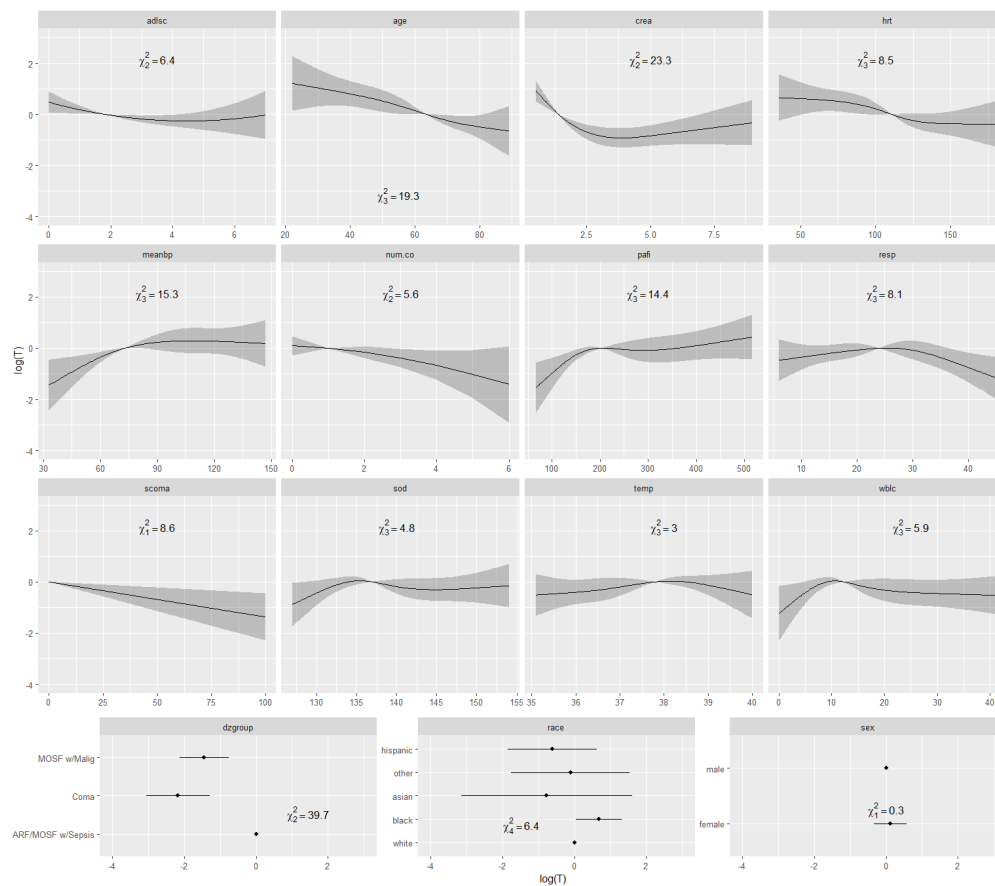


```
k<-4
f<- psm(S~ rcs(age,k)+ sex + dzgroup + pol ( num.co ,2)+ scoma +
  pol (adlsc,2)+ race + rcs( meanbp ,k)+ rcs (hrt ,k) +
  rcs (resp ,k)+ rcs ( temp ,k)+ rcs (crea ,3)+ rcs (sod ,k )+
  rcs ( wblc,k )+ rcs ( pafi,k), dist = "lognormal" )
plot ( anova (f), main="partial chi-square stastitics")
```

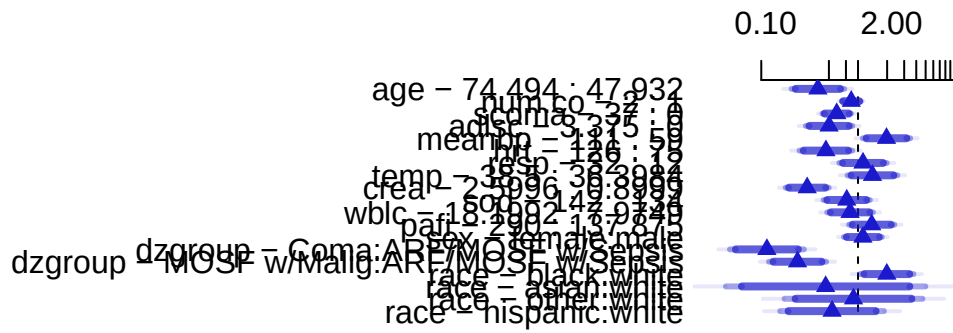

partial chi-square stastitics



```
a<- anova(f)
#ggplot ( rms::Predict(f , ref.zero = TRUE ), vnames = 'names' ,
#        sepdiscrete = 'vertical' , anova =a)
```



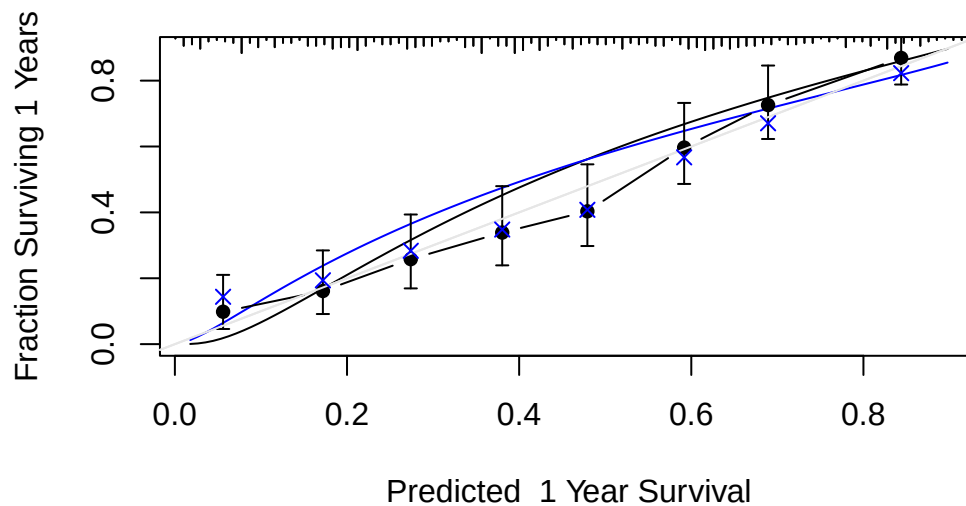
```
plot ( summary (f), log =TRUE , main = '' )
```



```
g <- stats::update (f, x=TRUE, y=TRUE )
set.seed (717)
validate(g,B=300)
```

	index.orig	training	test	optimism	index.corrected	n
Dxy	0.5095	0.5398	0.4734	0.0664	0.4432	300
R2	0.6356	0.7111	0.5532	0.1579	0.4777	300
Intercept	0.0000	0.0000	-0.0430	0.0430	-0.0430	300
Slope	1.0000	1.0000	0.8813	0.1187	0.8813	300
D	0.5208	0.6060	0.4372	0.1688	0.3520	300
U	-0.0044	-0.0044	-0.0145	0.0101	-0.0145	300
Q	0.5251	0.6104	0.4518	0.1587	0.3665	300
g	2.0848	2.1943	1.9517	0.2426	1.8422	300

```
set.seed (717)
cal <- calibrate (g , u=1, B =300)
plot (cal , subtitles = FALSE )
set.seed (717)
cal <- calibrate (g , cmethod = 'KM' , u=1, m=60 , B=120 , pr= FALSE )
plot (cal , add = TRUE )
```



생존예측하기

$$S(t) = P_r(T > t)$$

$S(t)$: 시간 t 까지 주어진 사건(사망)이 발생하지 않을 확률

T : 사건이 발생하기까지의 시간

$S(t)$ 의 값은 0과 1사이에 있으며, 시간이 지남에 따라 감소함 센서링 : 관찰기간 동안 사건이 발생하지 않은 경우

위험함수 (Hazard Function)

$$\text{hazard function } h(t) = \lim_{\Delta t \rightarrow 0} \frac{\Pr[(t \leq T < t + \Delta t | T \geq t)]}{\Delta t} = \frac{p(t)}{S(t)}$$

II.D.2 다변량 분석

Cox Proportional Hazard model

- **이론적 내용**: 콕스 모델은 시간에 따라 변화하는 위험을 모델링하는 반모수적 방법이다. 여러 설명 변수들이 생존 시간에 미치는 상대적 위험도를 추정하며, 변수 간의 상호작용 및 시간에 따른 영향도 고려할 수 있다.
- **근거**: 모델은 생존 시간에 영향을 미치는 변수들의 비례 위험 가정을 바탕으로 한다.
- **활용 예**: 다변량 생존 분석, 위험 요인 분석, 약물의 효과 분석 등.

Parametric survival model

- **이론적 내용**: 파라메트릭 모델은 생존 시간이 특정 확률 분포(예: 지수 분포, 웨이블 분포)를 따른다고 가정한다. 이 모델은 생존 시간의 분포와 관련된 매개변수를 추정하며, 정확한 분포 가정 하에 높은 효율성과 예측력을 제공한다.
- **근거**: 분포 가정은 데이터의 특성과 이전 연구 결과를 바탕으로 결정된다.
- **활용 예**: 생존 시간의 정확한 예측, 실험 계획 및 위험 평가, 고장률 분석 등.
- rms package에서는 binary, polytomous, spline, ordinal 등의 인자들과 섞어서 사용 가능함 - 다만 기존의 회귀모델에서 사용하는 회귀효과 $X\beta = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k$ 와 survival function $\lambda(t|X)$ 는 다음과 같은 관계가 있다.

$$\lambda(t|X) = \lambda(t) \cdot e^{X\beta}$$

여기서 $\lambda(t)$ 는 underlying hazard function 이라고 함 : $\beta = 0$

Dataset

data cleaning이 끝난 데이터는 Rdata format으로 저장하는 것이 좋다.

- 왜? : R 내부의 데이터구조를 그대로 간직하고 있으면서 가장 메모리를 적게 차지하는 형태이기 때문에 data cleaning 내용 :

1. 이름이 date로 끝나는 변수들은 전부 POSIX.ct 포맷으로 변경하고,
2. AJCC7 stage 이름에서 최종 stage group만 뽑아내서 AJCC7s 변수에 넣기
3. opdate와 last_fudate 의 차이를 계산하여 survival 변수에 넣기 (month로 계산하여 반환됨)

```
library(RDS)
S<-readRDS("S.rds")
str(S)
```

```
'data.frame': 1606 obs. of 33 variables:
 $ hospital      : chr  "KCCH" "KCCH" "KCCH" "KCCH" ...
 $ ChartNo       : int   637999 833629 980412 854342 1019918 821397 938358 1078360 811394 774217 ...
 $ Ca_Dx_date    : chr   "" "2000-07-07" "2007-12-06" "" ...
 $ Sex           : chr   "M" "M" "M" "M" ...
 $ OpDate        : chr   "1999-12-28" "2000-07-19" "2009-12-01" "2001-11-27" ...
 $ OpAge         : num    64 73 57 73 74 64 51 55 73 64 ...
 $ Birthdate     : chr   "1935-03-25" "1926-11-26" "1951-12-02" "1928-09-15" ...
 $ NeoAdj        : int    0 0 0 0 0 0 1 1 0 0 ...
 $ optype        : int    1 1 1 1 1 1 1 1 1 3 ...
 $ VATS          : int    0 0 0 0 1 0 0 0 0 0 ...
 $ Ctsolid_size  : num    1.5 3 3.6 4.5 3.8 7.5 0 0 0 0.7 ...
 $ CT_size       : num    1.5 3 3.6 4.5 3.8 7.5 0 0 0 0.7 ...
 $ path_size     : num    1.5 3 4 4 4.4 7.5 0 0 0 0.7 ...
 $ Cell_type     : int    2 2 2 2 2 2 1 2 4 2 ...
 $ Patho         : chr   "Sq" "Sq" "Sq" "Sq" ...
 $ survival      : int    77 11 35 64 45 69 73 15 0 1 ...
 $ rec_free      : num    34 8 23 64 45 33 73 15 0 1 ...
 $ Recur_date    : chr   "2002-11-11" "2001-04-16" "2011-11-02" "" ...
 $ recurrence    : int    1 1 1 0 0 1 0 0 0 0 ...
 $ Op_mortality  : int    0 0 0 0 0 0 0 0 1 1 ...
 $ death         : num    1 1 0 1 0 1 0 0 1 1 ...
 $ Death_date    : chr   "2006-06-15" "2001-06-30" "" "2007-04-28" ...
 $ last_fu       : chr   "2006-06-15" "2001-06-30" "2012-11-30" "2007-04-28" ...
 $ AJCC7         : chr   "T2NOM0 stageIB" "T2aNOM0 Stage IB" "T2aNOM0 Stage IB" "T2NOM0 stageIB" ...
 $ Tstage        : chr   "2a" "2a" "2a" "2a" ...
 $ Nstage        : int    0 0 0 0 0 0 0 0 0 0 ...
 $ Mstage        : chr   "0" "0" "0" "0" ...
 $ Differentiation: chr   "MD" "MD" "MD" "WD" ...
 $ Adj_RT        : int    0 1 0 0 0 0 0 0 0 1 ...
 $ adj_CT        : int    1 0 0 0 0 1 1 0 0 0 ...
 $ survival.60   : num    1 0 NA 1 NA 1 1 NA 0 0 ...
 $ type          : Factor w/ 32 levels "AnyT AnyN M1a",...: 20 13 13 20 13 20 23 23 23 23 ...
 $ stage         : Factor w/ 8 levels "Stage 0","Stage IA",...: 3 3 3 3 3 3 5 5 5 5 ...
```

```
colnames(S)[23] <- "last_fudate"
pattern_stage <- "[Ss]tage[ ]?([\\w]{1,3}[AB]?)"
S$AJCC7s <- str_match( S$AJCC7, pattern_stage )[,2]
pattern_date <- "[Dd]ate$"
var_date <- str_match(colnames(S), pattern_date)
colnames(S) [str_detect(colnames(S), pattern_date)]
```

```
[1] "Ca_Dx_date" "OpDate" "Birthdate" "Recur_date" "Death_date"
[6] "last_fudate"
```

```

var_date <- colnames(S) [str_detect(colnames(S), pattern_date)]
for (i in var_date) {
  S[, i] <- ymd(S[,i] , tz="Asia/Seoul")
}
S$survival <-
round(difftime(S$last_fudate,S$OpDate, units = "day") /365.24219 *12,0)
attr(S$survival, "units") <- "months"
rec_free <-
round(difftime(S$Recur_date,S$OpDate, units = "day") /365.24219 *12,0)
attr(rec_free, "units") <- "months"
rec_free[is.na(rec_free)] <- S$survival[is.na(rec_free)]
S$rec_free <- rec_free
str(S)

```

```

'data.frame': 1606 obs. of 34 variables:
 $ hospital      : chr  "KCCH" "KCCH" "KCCH" "KCCH" ...
 $ ChartNo       : int   637999 833629 980412 854342 1019918 821397 938358 1078360 811394 774217 ...
 $ Ca_Dx_date    : POSIXct, format: NA "2000-07-07" ...
 $ Sex           : chr   "M" "M" "M" "M" ...
 $ OpDate        : POSIXct, format: "1999-12-28" "2000-07-19" ...
 $ OpAge         : num    64 73 57 73 74 64 51 55 73 64 ...
 $ Birthdate     : POSIXct, format: "1935-03-25" "1926-11-26" ...
 $ NeoAdj        : int     0 0 0 0 0 0 1 1 0 0 ...
 $ optype        : int     1 1 1 1 1 1 1 1 3 ...
 $ VATS          : int     0 0 0 0 1 0 0 0 0 0 ...
 $ Ctsolid_size  : num     1.5 3 3.6 4.5 3.8 7.5 0 0 0 0.7 ...
 $ CT_size       : num     1.5 3 3.6 4.5 3.8 7.5 0 0 0 0.7 ...
 $ path_size     : num     1.5 3 4 4 4.4 7.5 0 0 0 0.7 ...
 $ Cell_type     : int     2 2 2 2 2 2 1 2 4 2 ...
 $ Patho         : chr    "Sq" "Sq" "Sq" "Sq" ...
 $ survival      : 'difftime' num   78 11 36 65 ...
 ..- attr(*, "units")= chr "months"
 $ rec_free      : 'difftime' num   34 9 23 65 ...
 ..- attr(*, "units")= chr "months"
 $ Recur_date    : POSIXct, format: "2002-11-11" "2001-04-16" ...
 $ recurrence    : int     1 1 1 0 0 1 0 0 0 0 ...
 $ Op_mortality  : int     0 0 0 0 0 0 0 0 1 1 ...
 $ death         : num     1 1 0 1 0 1 0 0 1 1 ...
 $ Death_date    : POSIXct, format: "2006-06-15" "2001-06-30" ...
 $ last_fudate   : POSIXct, format: "2006-06-15" "2001-06-30" ...
 $ AJCC7        : chr    "T2NOM0 stageIB" "T2aNOM0 Stage IB" "T2aNOM0 Stage IB" "T2NOM0 stageIB" ...
 $ Tstage       : chr    "2a" "2a" "2a" "2a" ...
 $ Nstage       : int     0 0 0 0 0 0 0 0 0 0 ...
 $ Mstage       : chr    "0" "0" "0" "0" ...
 $ Differentiation: chr    "MD" "MD" "MD" "WD" ...
 $ Adj_RT       : int     0 1 0 0 0 0 0 0 0 1 ...
 $ adj_CT       : int     1 0 0 0 0 1 1 0 0 0 ...

```

```
$ survival.60      : num  1 0 NA 1 NA 1 1 NA 0 0 ...
$ type             : Factor w/ 32 levels "AnyT AnyN M1a",...: 20 13 13 20 13 20 23 23 23 23 ...
$ stage            : Factor w/ 8 levels "Stage 0","Stage IA",...: 3 3 3 3 3 3 5 5 5 5 ...
$ AJCC7s           : chr  "IB" "IB" "IB" "IB" ...
```

Factor & Data cleaning

parametric survival model을 위해서는 parametric variable vs non-parametric variable을 구분해야 함

```
library(magrittr)
S[,c("optype","Tstage")] %>% table()
```

```
      Tstage
optype 1a  1b  2a  2b  2q   3   4  is
1    140 158 534 150   0 195  51   3
2     17   6  22   3   0   7   4   0
3      6  14  96  30   1 108  61   0
```

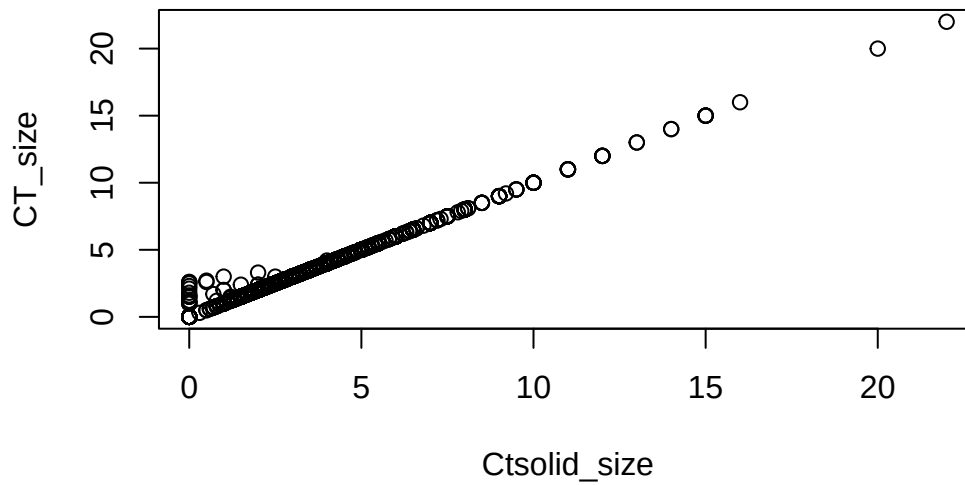
```
S$optypef <- factor(S$optype,
                    labels = c("lobectomy","sleeve lobectomy","pneumonectomy"))
S$NeoAdjf <- factor(S$NeoAdj, labels = c(FALSE, TRUE))
S$Nstage <- as.character(S$Nstage)
lung.df<-S
table(lung.df$Differentiation)
```

```
#VALUE!      MD      n      PD      WD
      7      706     410     279     204
```

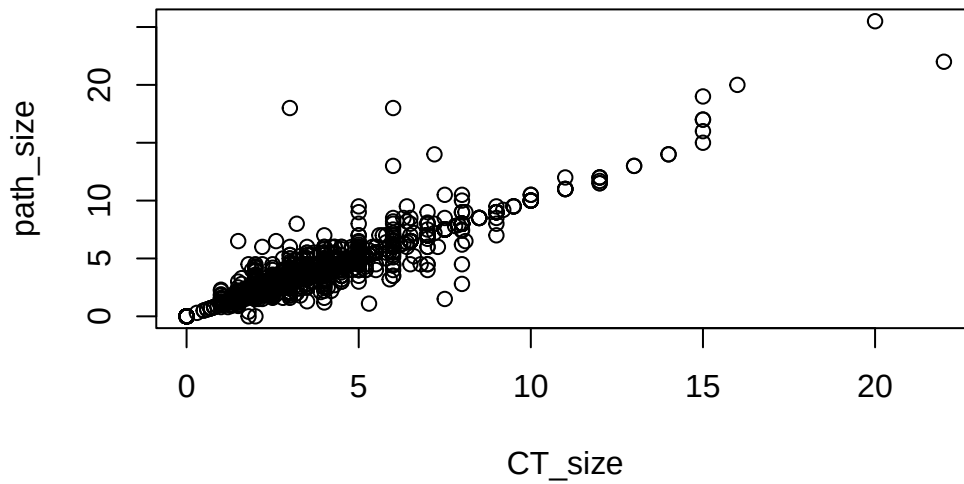
```
lung.df[lung.df$Differentiation=="#VALUE!","Differentiation"] <- "n"
table(lung.df$Differentiation)
```

```
MD    n  PD  WD
706 417 279 204
```

```
lung.df$Adj_RT <- factor(lung.df$Adj_RT, labels = c("No","Yes"))
lung.df$adj_CT <- factor(lung.df$adj_CT, labels = c("No","Yes"))
lung.df$NeoAdj <- factor(lung.df$NeoAdj, labels = c("No","Yes"))
lung.df$VATS   <- factor(lung.df$VATS, labels = c("No","Yes"))
plot(lung.df[,c("Ctsolid_size","CT_size")])
```



```
plot(lung.df[,c("CT_size", "path_size")])
```



psm 모델에서는 survival period는 0 이하의 값을 가지면 안됨

op mortality 때문에 생기는 문제 - op mortality를 케이스에서 제거하거나 1을 전부 더해서 해결

```
table(lung.df$survival==0)
```

```
FALSE TRUE
1585    21
```

```
table(lung.df$survival<0)
```

```
FALSE
1606
```

```
lung.df$survival <- lung.df$survival+1
```

변수 선택